Intelligence Artificielle Guidée par les Lois Scientifiques

Comment apprendre aux modèles pilotés par les données à respecter les lois qui gouvernent l'univers

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyb er Sécurité



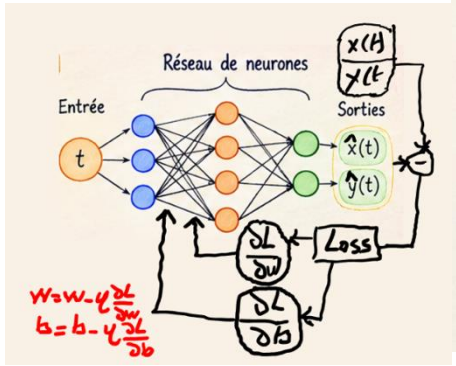
https://www.youtube.com/watch?v=jq04Mkid5s0

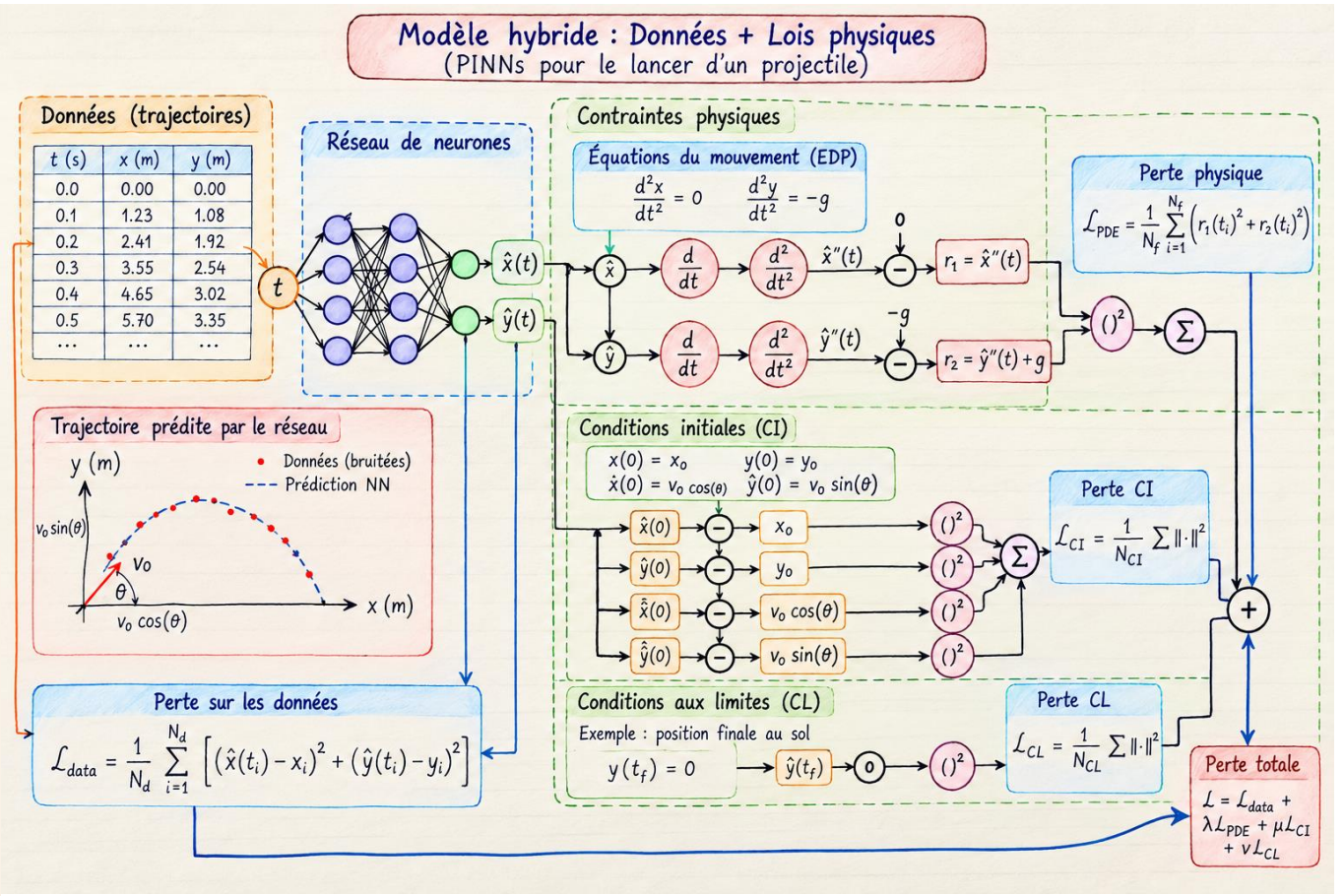
https://github.com/mohamedYoussfi/Conf-ICAIS-26-PINNs



PyTorch

python





Intelligence Artificielle

l'IA qui raisonne à partir de **règles explicites**

l'IA qui apprend à partir de **données, d'expérience ou de contraintes physiques.**

IA Symbolique
Approche historique et toujours utilisée

Une approche de l'IA qui
- représente les connaissances sous forme de symboles, de règles explicites et de logique formelle.
- Adaptée aux systèmes déterministes
- Utilisée dans systèmes experts et les moteurs de règles
- IA Explicable, contrôlable et prévisible

IA Basée sur l'apprentissage (Machine Learning)
Approche très dominante

IA Prédictive et décisionnelle

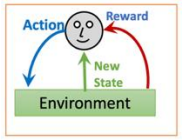
Apprendre pour prédire et prendre des décisions à partir des **données structurées ou non structurées,** de l'**expérience** **ou de** **la physique**

Modèles Pilotés par les données
- Apprentissage Supervisé
- Apprentissage Non Supervisé

Besoin de données en quantité et en qualité

Algorithmes :
- SVM, DT, RF, KNN, XGBoost, etc.
- Neural Network (ANN, CNN, RNN, LSTM)

Modèles Pilotés par l'expérience
- Apprentissage Par renforcement



Besoin une abstraction de l'environnement

Algorithmes :
- Q-learning, Deep Q-learning
- PPO
- Neural Network

Modèles Guidés par les lois de la physique
- PINN

Besoin des lois de la, physique et des données

Algorithmes :
- Neural Network

IA Générative

Apprend les structures des données, souvent non structurées, pour générer de nouveaux contenus réalistes : texte, image, son, vidéo, code.
Apprentissage Auto Supervisé, LLMs

Algorithmes :
- Neural Network (Transformers, GAN, …)

IA Agentique

Architecture dans laquelle un agent **perçoit son environnement**, **raisonne**, **planifie**, **utilise des outils** et **agit** pour atteindre un objectif.

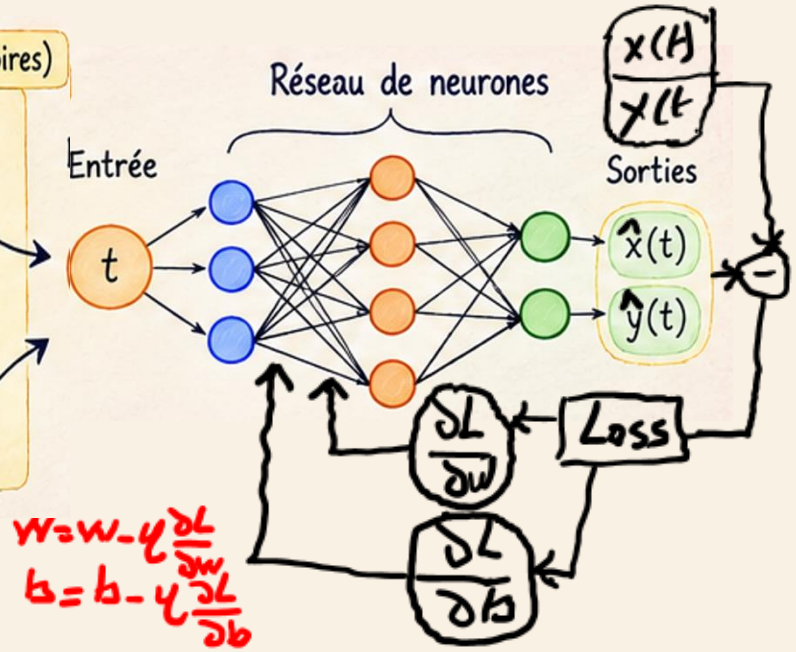
Technologies :
- LLM (Neural Network)
- Tools, MCP
- Planification, RAG,
- Apprentissage par renforcement

Contexte et Problématique

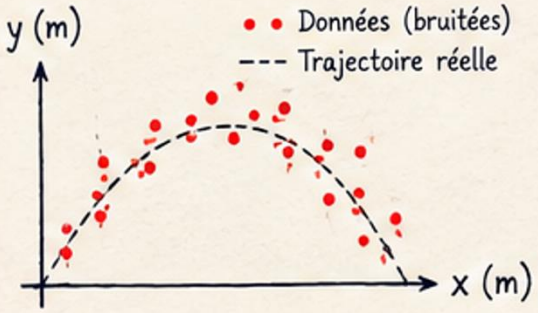
Data Driven Model
Artificial Neural Network

Dataset (données de trajectoires)

t (s)	x (m)	y (m)
0.0	0.00	0.00
0.1	1.23	1.08
0.2	2.41	1.92
0.3	3.55	2.54
0.4	4.65	3.02
0.5	5.70	3.35
...	...	...



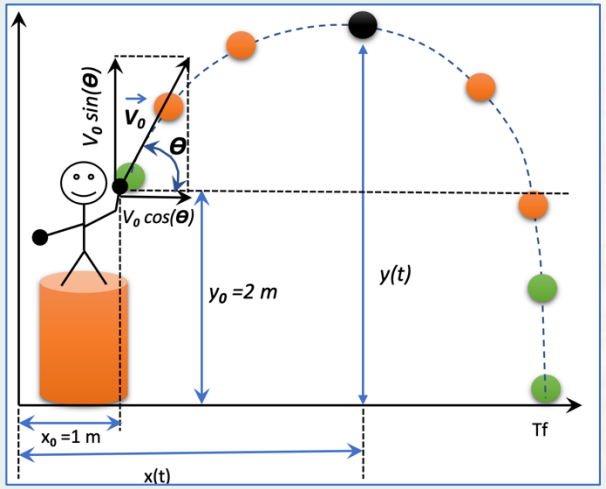
Données bruitées



Limites du modèle data-driven

- Sensible au bruit dans les données
- Nécessite beaucoup de données
- Risque de sur-apprentissage
- Modèle biaisé (peut ne pas respecter les lois physiques)
- Mauvaise généralisation (hors distribution)

Physics Driven Model



Équations différentielles (EDP)

$$\frac{d^2x}{dt^2} = 0 \qquad \frac{d^2y}{dt^2} = -g$$

g : accélération de la pesanteur ($\approx 9.81\ m/s^2$)

Conditions initiales (CI)

$x(0) = x_0 \qquad y(0) = y_0$

$\dot{x}(0) = v_0 \cos\theta \qquad \dot{y}(0) = v_0 \sin\theta$

Conditions aux limites (CL)

Exemple : sol $\quad y(t_f) = 0$

$$x(t) = x_0 + v_0 \cos(\theta)\, t \qquad y(t) = y_0 + v_0 \sin(\theta)\, t - \frac{1}{2} g t^2$$

t → **Physics Model** → $x(t)$, $y(t)$

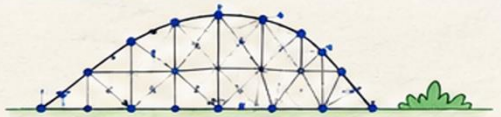
Hypothèses simplificatrices

- Gravité seule (sans frottement de l'air)
- Terrain plat
- Masse constante
- Pas d'effets de rotation, vent, etc.



Résolution numérique

- Méthodes analytiques (solutions simples)
- Méthodes numériques (ex. éléments finis)



- Calculs intensifs pour des problèmes complexes
- Géométries et milieux hétérogènes
- Non-linéarités, couplages multiphysiques, etc.

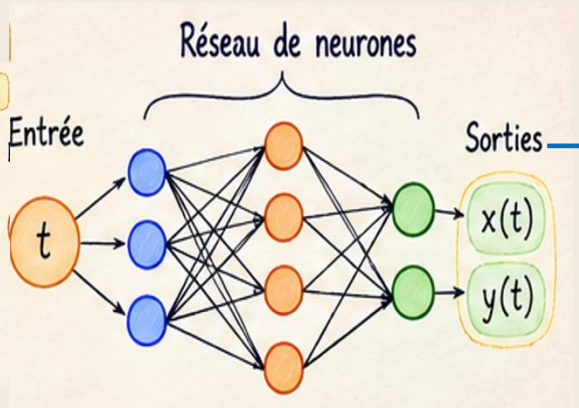
Limites du modèle physique

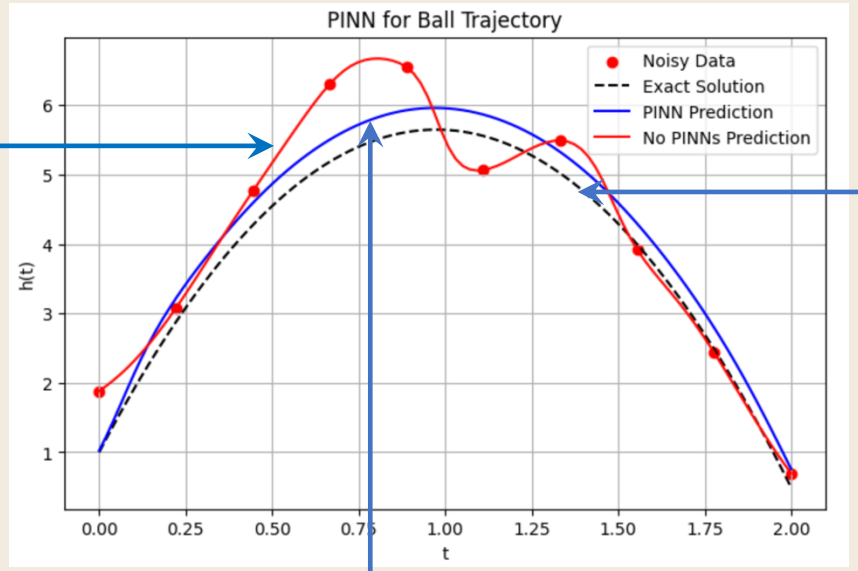
- Dépend des hypothèses simplificatrices
- Peut être difficile à résoudre (géométries complexes)
- Ne prend pas en compte certains phénomènes réels

Problématique : Vers des modèles hybrides informés par la physique

Dataset (données de trajectoires)

t (s)	x (m)	y (m)
0.0	0.00	0.00
0.1	1.23	1.08
0.2	2.41	1.92
0.3	3.55	2.54
0.4	4.65	3.02
0.5	5.70	3.35
...	...	...





Équations différentielles (EDP)

$$\frac{d^2x}{dt^2} = 0 \qquad \frac{d^2y}{dt^2} = -g$$

g : accélération de la pesanteur ($\approx 9.81\ m/s^2$)

Conditions initiales (CI)

$x(0) = x_0 \qquad y(0) = y_0$

$\dot{x}(0) = v_0 \cos\theta \qquad \dot{y}(0) = v_0 \sin\theta$

Conditions aux limites (CL)

Exemple : sol $\quad y(t_f) = 0$

t → **Data Model** → x(t), y(t)

t → **Physics Model** → x(t), y(t)

Hybrid Model (PINN)
Data Driven + Physics Informed

Dataset (données de trajectoires)

t (s)	x (m)	y (m)
0.0	0.00	0.00
0.1	1.23	1.08
0.2	2.41	1.92
0.3	3.55	2.54
0.4	4.65	3.02
0.5	5.70	3.35
...	...	...

Réseau de neurones — Entrée t — Sorties x(t), y(t)

Équations différentielles (EDP)

$\frac{d^2x}{dt^2} = 0 \qquad \frac{d^2y}{dt^2} = -g$

g : accélération de la pesanteur ($\approx 9.81\ m/s^2$)

Conditions initiales (CI)

$x(0) = x_0 \qquad y(0) = y_0$

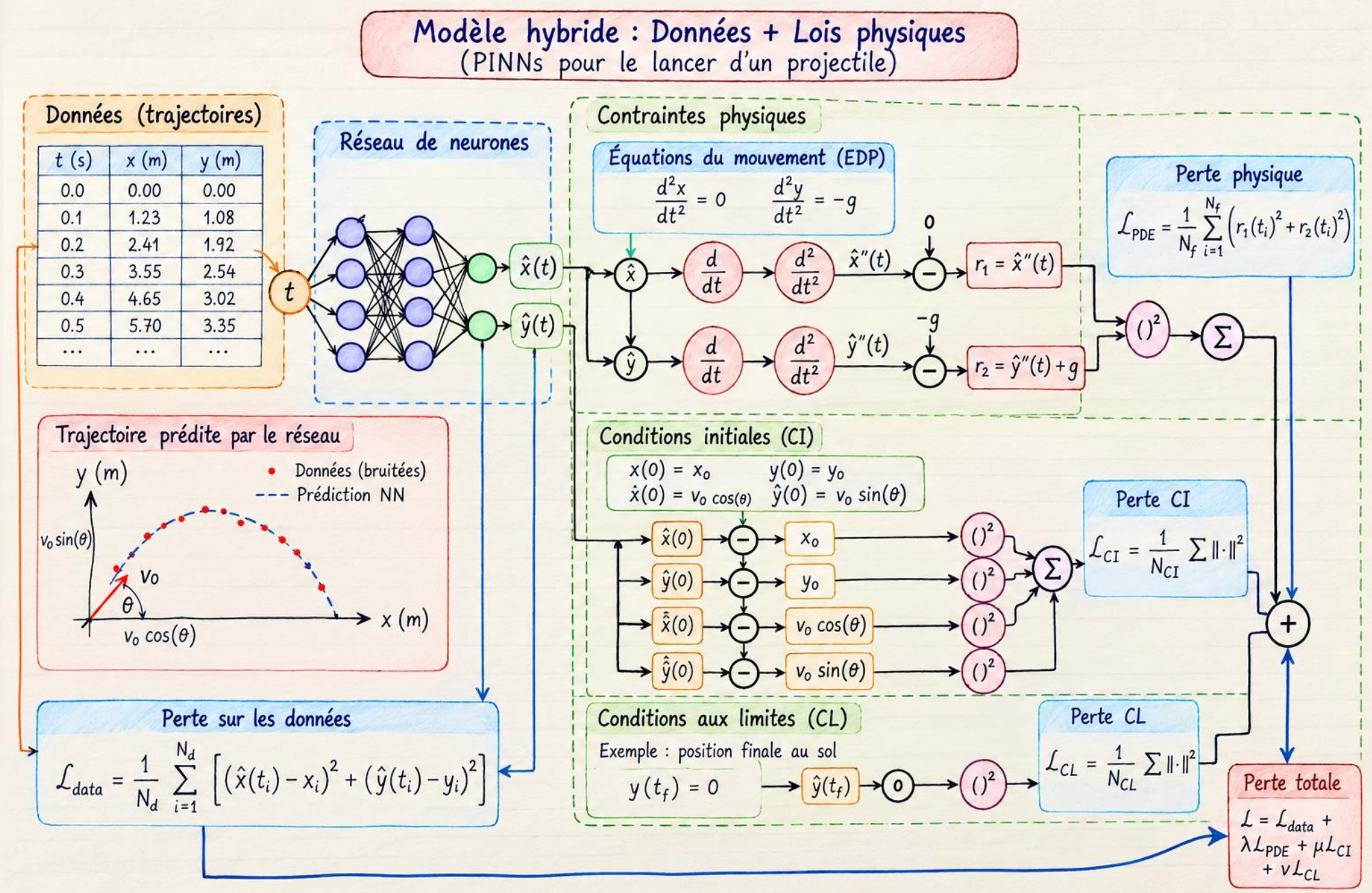
$\dot{x}(0) = v_0 \cos\theta \qquad \dot{y}(0) = v_0 \sin\theta$

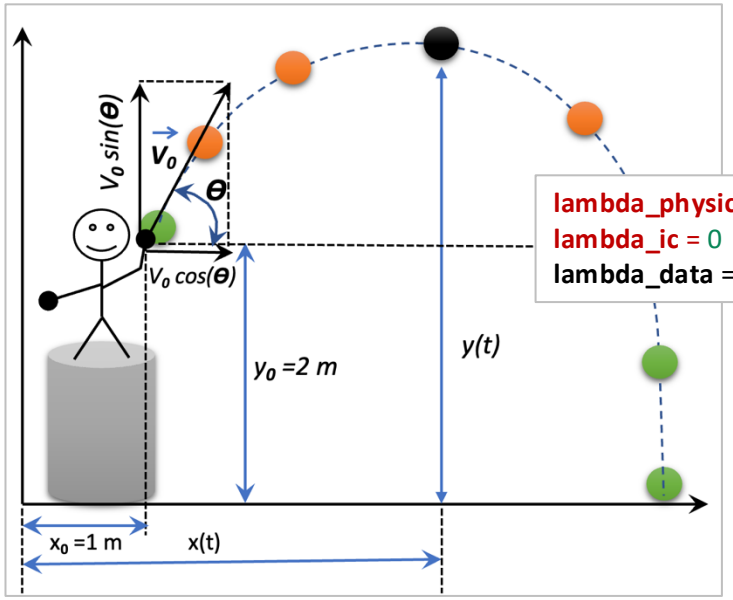
Conditions aux limites (CL)

Exemple : sol $\quad y(t_f) = 0$

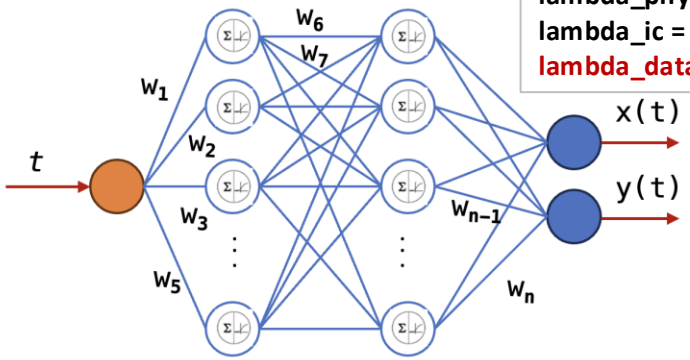
$$\mathcal{L}_{PINN} = \lambda_{data}\mathcal{L}_{data} + \lambda_{physics}\mathcal{L}_{physics} + \lambda_{IC}\mathcal{L}_{IC}$$

Problématique : Vers des modèles hybrides informés par la physique





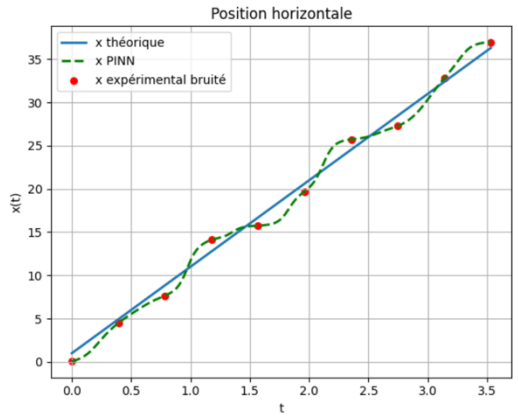
lambda_physics = 0
lambda_ic = 0
lambda_data = 1

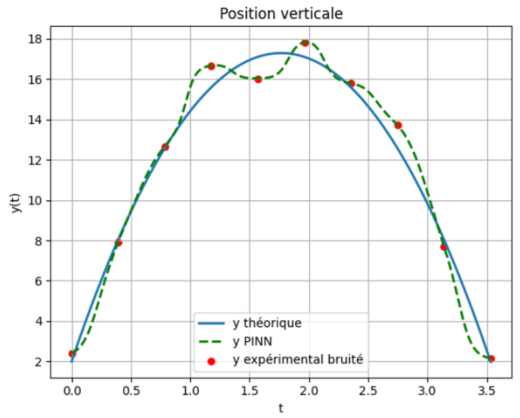


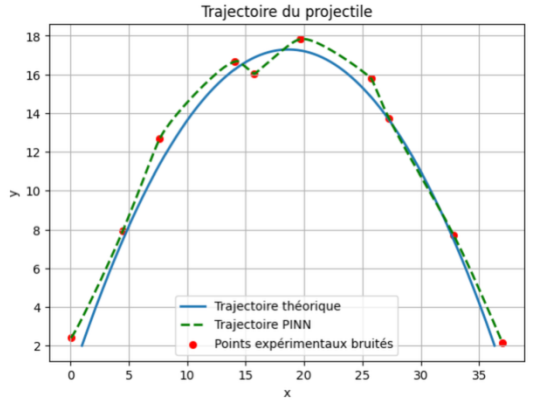
lambda_physics = 1
lambda_ic = 1
lambda_data = 0

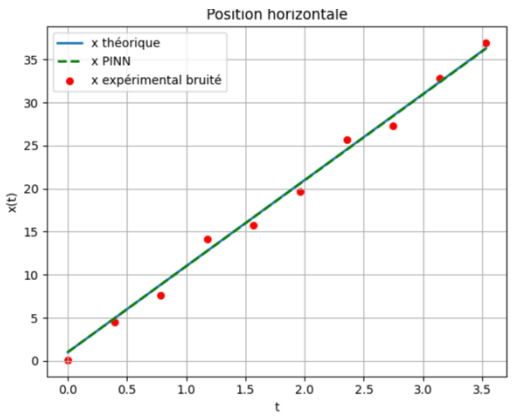
$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

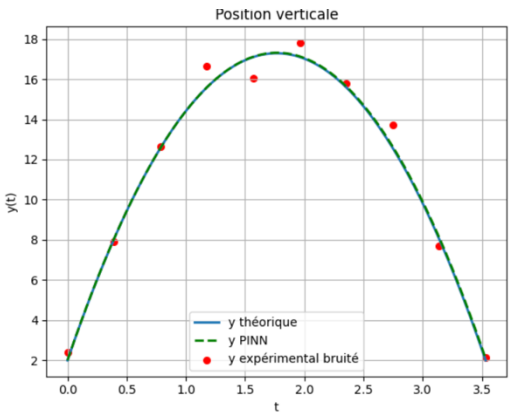
lambda_physics = 1
lambda_ic = 1
lambda_data = 1

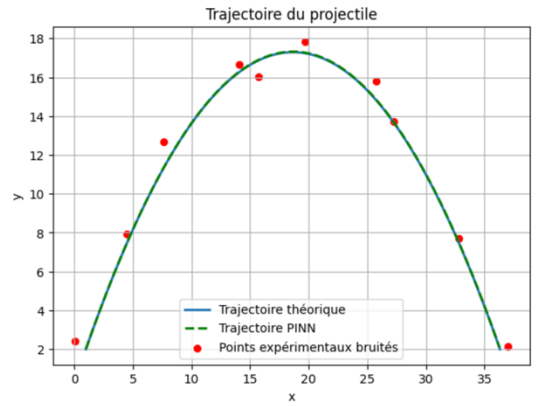


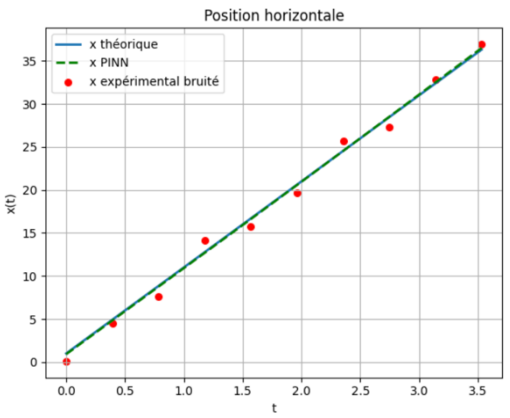


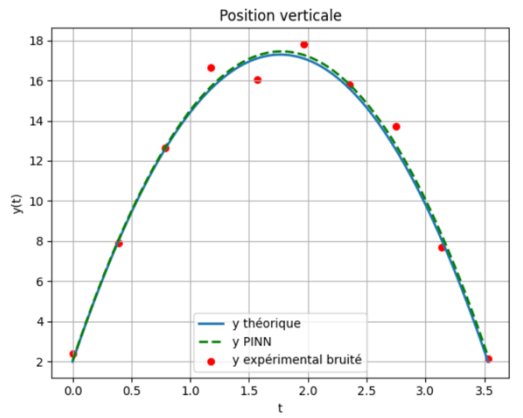


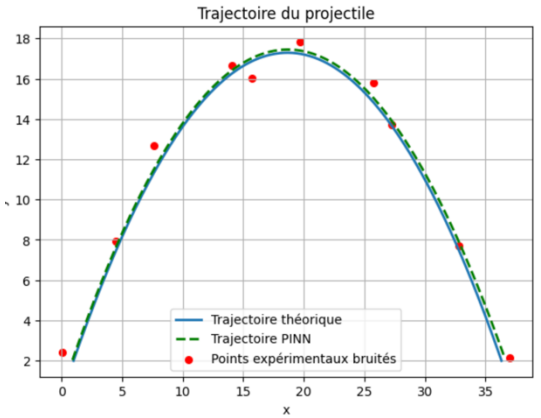




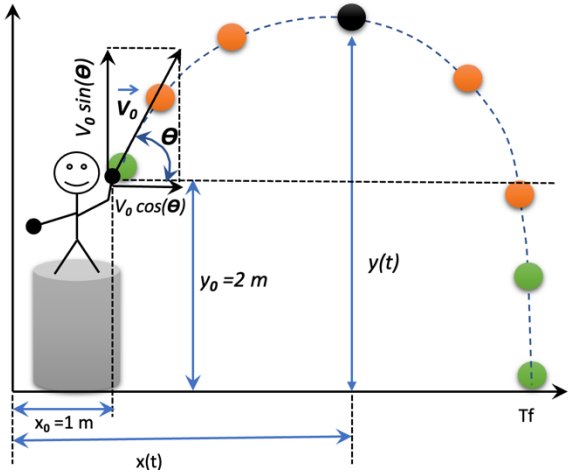








Physics based Model



$$\begin{cases} \dfrac{d^2x(t)}{dt^2} = 0 \\ \dfrac{d^2y(t)}{dt^2} = -g \end{cases}$$

$$\begin{cases} x(0) = 1 \\ y(0) = 2 \\ \dfrac{dx}{dt}(0) = v_0\cos(\theta) \\ \dfrac{dy}{dt}(0) = v_0\sin(\theta) \end{cases}$$

$g = 9.81\ \mathrm{m/s^2}.$

$$x(t) = x_0 + v_0\cos(\theta)\,t$$

$$y(t) = y_0 + v_0\sin(\theta)\,t - \frac{1}{2}gt^2$$




```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math
import pandas as pd
torch.manual_seed(42)
# Paramètres physiques
g = 9.81
# Conditions initiales
x0 = 1.0
y0 = 2.0

teta = math.pi/3
v0 = 20 # m/s
vx0 = v0*math.cos(teta)
vy0 = v0*math.sin(teta)

def physics_model(t):
    # Solution théorique exacte
    x_exact = x0 + vx0 * t
    y_exact = y0 + vy0 * t - 0.5 * g * t ** 2
    return x_exact, y_exact

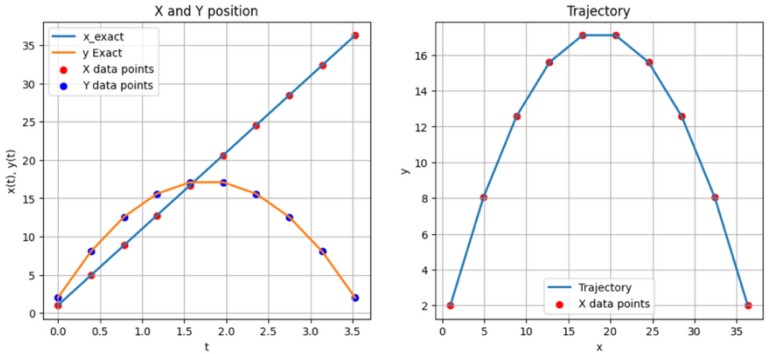
# Temps de vol théorique pour y = 2
T_final = 2 * vy0 / g
print("Temps de vol théorique :", T_final)
N_data = 10
t_test = torch.linspace(0, T_final, N_data).view(-1, 1)

x, y = physics_model(t_test)
```

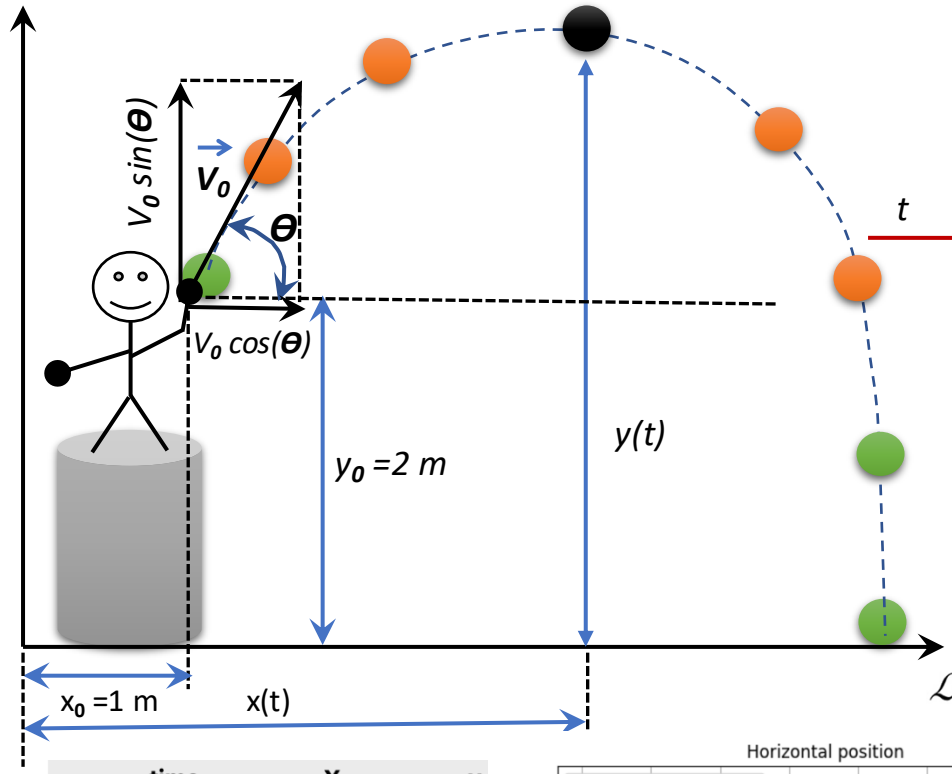
```
plt.figure(figsize=(12, 5))

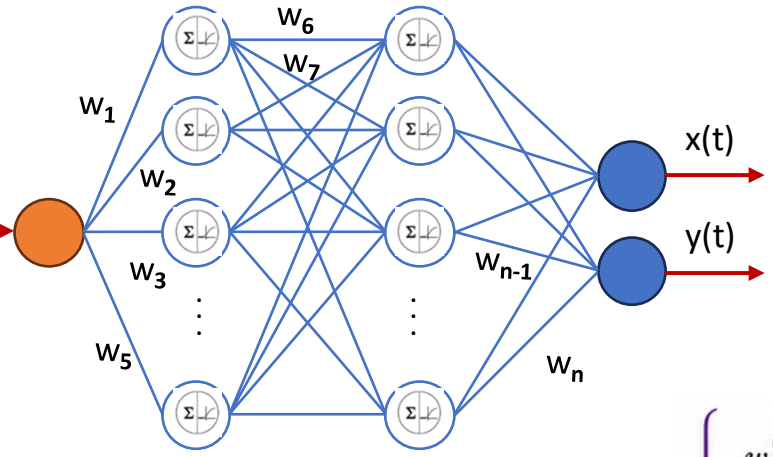
plt.subplot(1, 2, 1)
plt.plot(t_test, x, label="x_exact", linewidth=2)
plt.plot(t_test, y, label="y Exact", linewidth=2)
plt.scatter(t_test,x,color="red",label="X data points")
plt.scatter(t_test,y,color="blue",label="Y data points")
plt.xlabel("t")
plt.ylabel("x(t), y(t)")
plt.title("X and Y position")
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(x, y, label="Trajectory", linewidth=2)
plt.scatter(x,y,color="red",label="X data points")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectory")
plt.legend()
plt.grid()
```



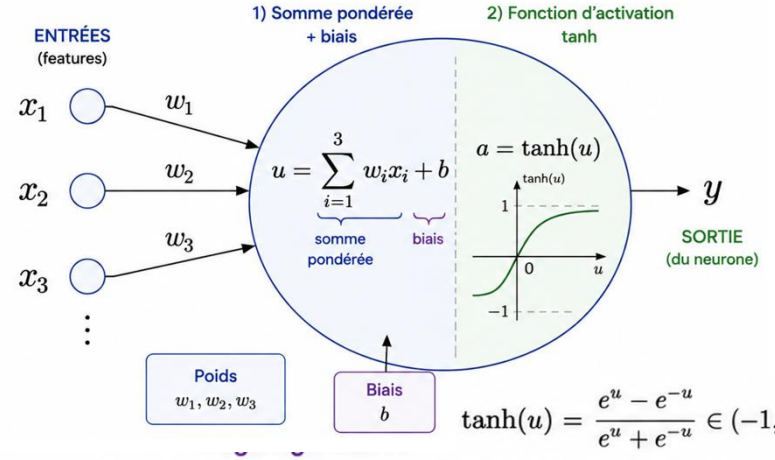
Data Driven Model : Neural Network





Hidden layer
64 neurones

Hidden layer
64 neurones



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

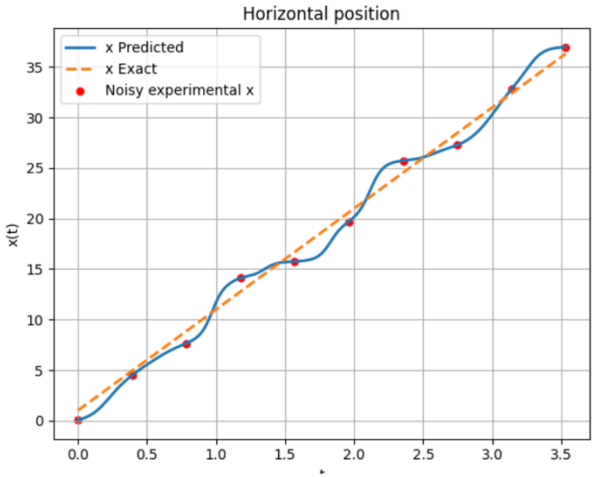
$$\tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} \in (-1, 1)$$

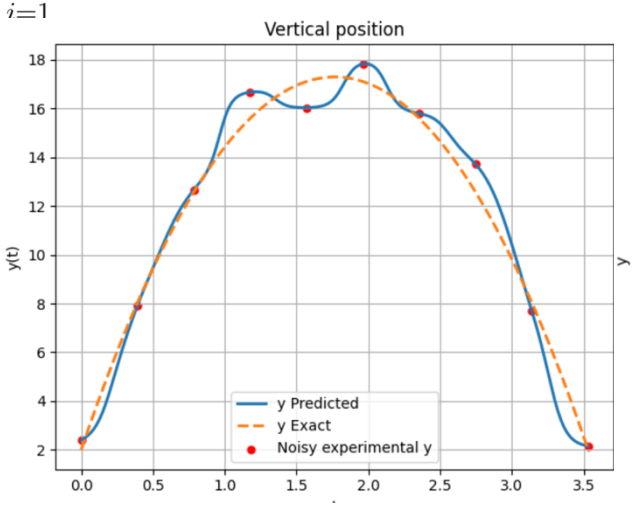
$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

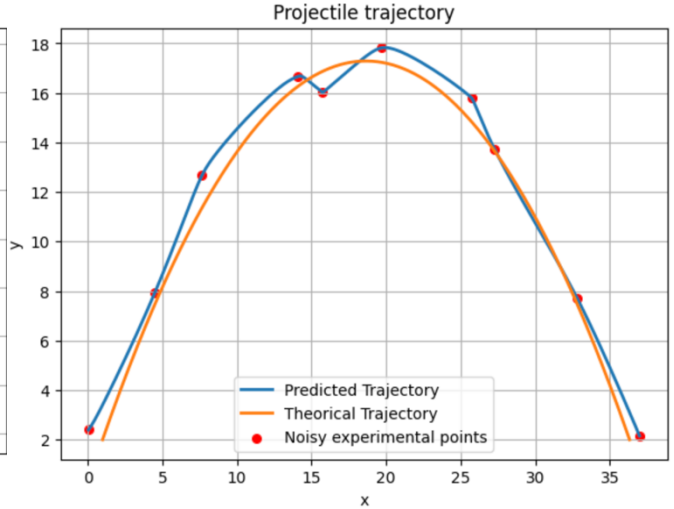
$$\begin{cases} w_i^{(k+1)} = w_i^{(k)} - \eta \dfrac{\partial \mathcal{L}}{\partial w_i^{(k)}} & \text{pour } i = 1, 2, 3 \\ b^{(k+1)} = b^{(k)} - \eta \dfrac{\partial \mathcal{L}}{\partial b^{(k)}} & \mathcal{L}_{data} = \dfrac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \end{cases}$$

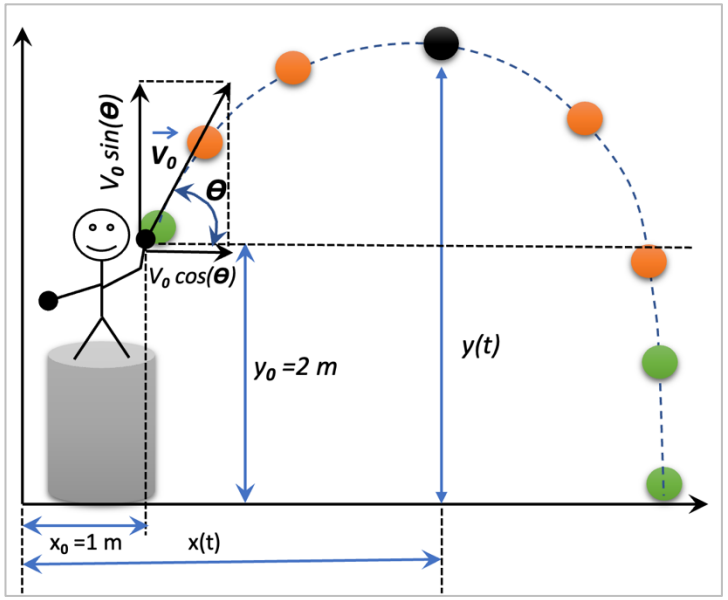
η : taux d'apprentissage (learning rate)

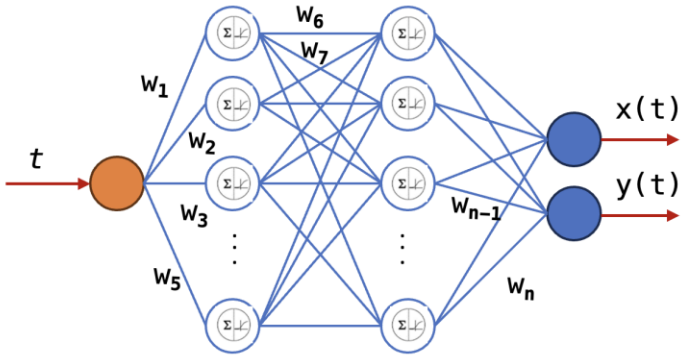
	time	X	y
0	0.000000	0.471534	2.219682
1	0.588532	9.586019	15.523645
2	1.177065	17.993155	25.836216
3	1.765597	28.226082	33.179218
4	2.354130	35.791260	35.383904
5	2.942662	44.638611	36.379066
6	3.531194	54.629513	32.692162
7	4.119727	62.136314	26.421669
8	4.708259	71.856407	15.405033
9	5.296792	80.820435	2.086739











$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
time = torch.tensor([
        [0.0000],
        [0.3924],
        [0.7847],
        [1.1771],
        [1.5694],
        [1.9618],
        [2.3541],
        [2.7465],
        [3.1388],
        [3.5312]]
       )
pos_exp = torch.tensor([
        [ 0.0488,  2.3954],
        [ 4.4880,  7.9184],
        [ 7.6540, 12.6630],
        [14.1065, 16.6683],
        [15.7570, 16.0307],
        [19.7154, 17.8220],
        [25.7322, 15.7916],
        [27.2776, 13.7168],
        [32.8069,  7.7049],
        [36.9753,  2.1562]]
      )
```

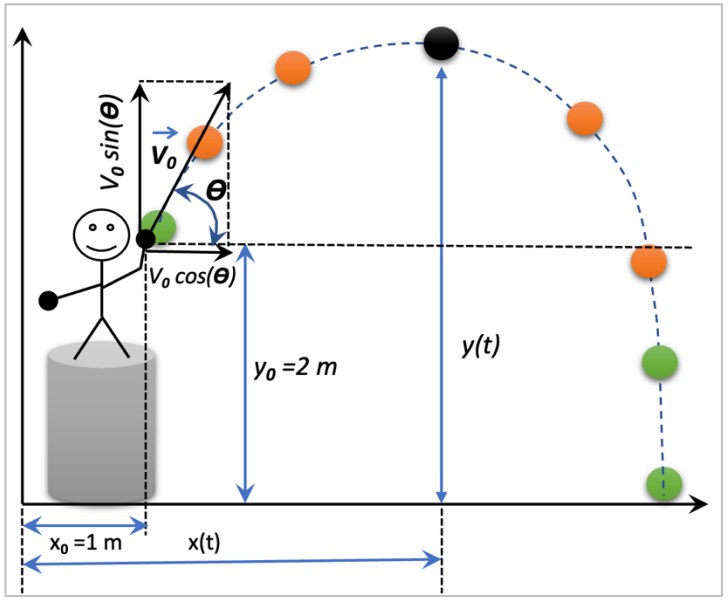
```
# Model
data_model = nn.Sequential(
    nn.Linear(1, 64),
    nn.Tanh(),
    nn.Linear(64, 2)  # sortie : [x(t), y(t)]
)
# Algo qui permet d'optimiser les paramètres du modèle
Optimizer = optim.Adam(data_model.parameters(), lr=0.001)
critetion = nn.MSELoss() # Mean Squared Error Loss

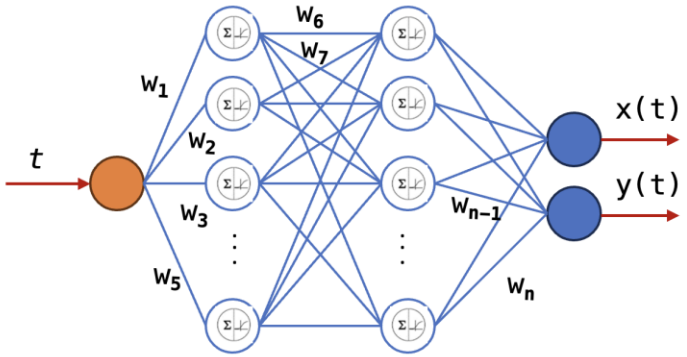
epochs = 8000

for epoch in range(epochs):
    model.train() # Positionner le modèle en mode train
    pred = data_model(time) # Prédiction
    x_pred = pred[:, 0:1] # Première colonne (x)
    y_pred = pred[:, 1:2] # Deuxième colonne (y)
    # Calculer le LOSS : Mean Squared Error
    loss_data = torch.mean( (pred - pos_exp) ** 2 )
    # loss_data = critetion(pred,position)
    optimizer.zero_grad() # Init gradient des params
    loss_data.backward() # d(loss)/dw et d(loss)/db

    optimizer.step() # Update weights : w = w – lr * dl/dw

    if (epoch + 1) % 1000 == 0:
        print(
         f"Epoch [{epoch+1}/{epochs}] "
         f"Loss: {loss_data.item():.6f} | "
         )
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

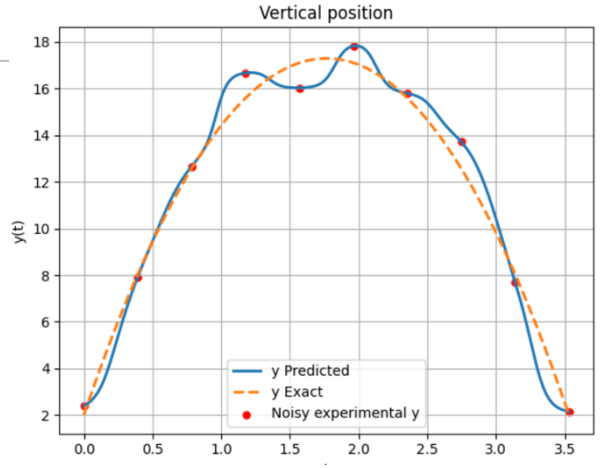
$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

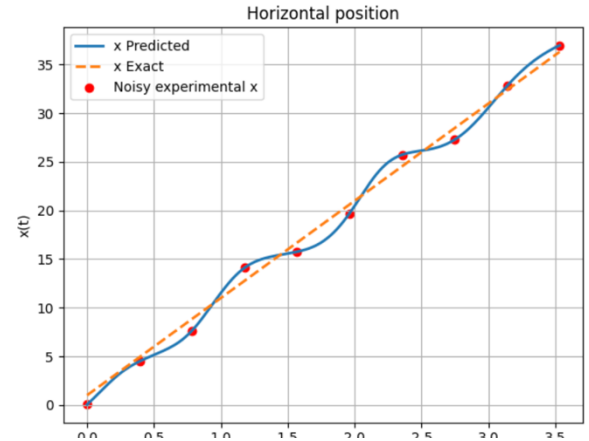
```
# Évaluation
t_final = 3.5312
t_test = torch.linspace(0, t_final, 300).view(-1, 1)
model.eval() # Positionner le modèle en mode Evaluation
with torch.no_grad():
   pred_test = data_model(t_test)

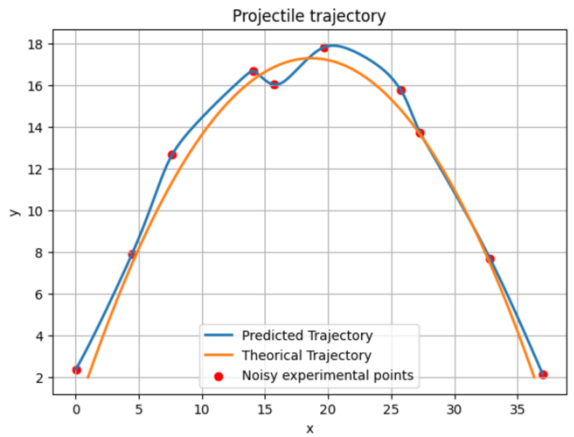
x_pred = pred_test[:, 0:1]
y_pred= pred_test[:, 1:2]

# Solution théorique exacte
x_exact = x0 + vx0 * t_test
y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2

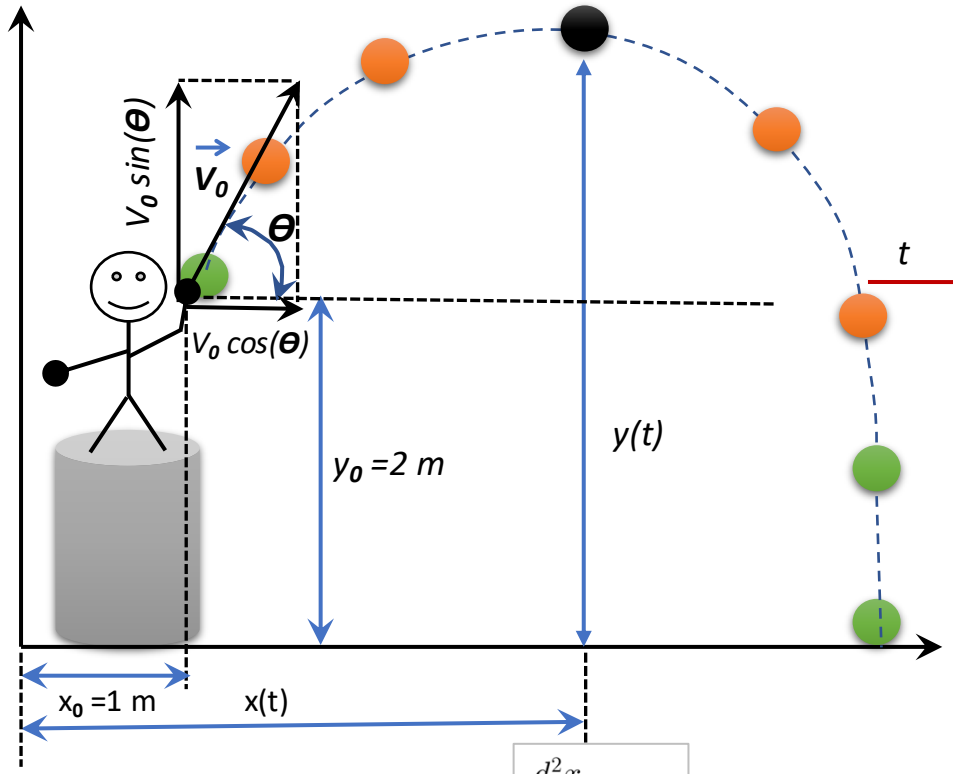

plt.subplot(1, 2, 2)
plt.plot(t_test, y_pred, label="y Predicted")
plt.plot(t_test, y_exact, "--", label="y Exact")
plt.scatter( time, pos_exp[:,1:2], color="red", s=25,label="Noisy exp y")
plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("Vertical position")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

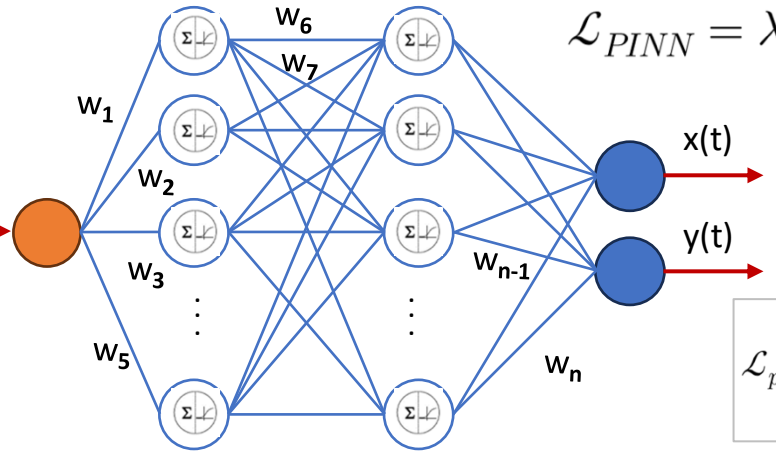






Physics Informed Neural Network





$$\mathcal{L}_{PINN} = \lambda_{data}\mathcal{L}_{data} + \lambda_{physics}\mathcal{L}_{physics} + \lambda_{IC}\mathcal{L}_{IC}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

$$\mathcal{L}_{physics} = \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\left(\frac{d^2\hat{x}(t_i)}{dt^2}\right)^2 + \left(\frac{d^2\hat{y}(t_i)}{dt^2} + g\right)^2\right]$$

$$\mathcal{L}_{IC} = (\hat{x}(0) - x_0)^2 + (\hat{y}(0) - y_0)^2 + \left(\frac{d\hat{x}}{dt}(0) - v_0\cos\theta\right)^2 + \left(\frac{d\hat{y}}{dt}(0) - v_0\sin\theta\right)^2$$

	time	X	y
0	0.000000	0.471534	2.219682
1	0.588532	9.586019	15.523645
2	1.177065	17.993155	25.836216
3	1.765597	28.226082	33.179218
4	2.354130	35.791260	35.383904
5	2.942662	44.638611	36.379066
6	3.531194	54.629513	32.692162
7	4.119727	62.136314	26.421669
8	4.708259	71.856407	15.405033
9	5.296792	80.820435	2.086739

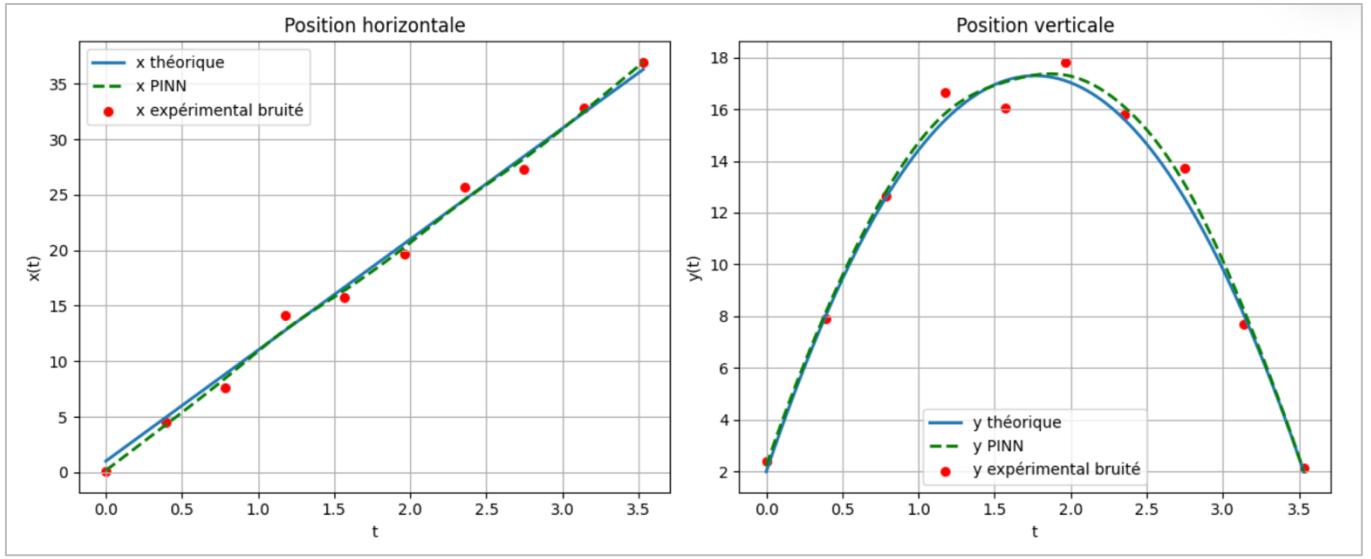
$$\frac{d^2x}{dt^2} = 0$$

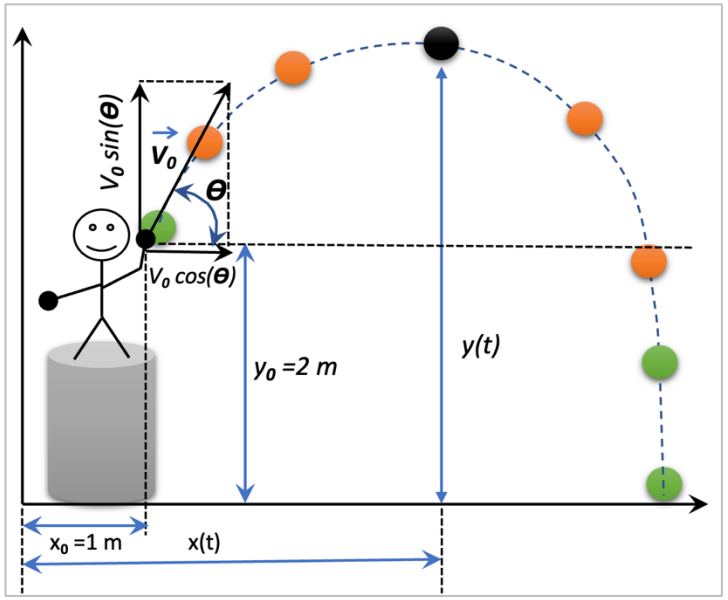
$$\frac{d^2y}{dt^2} = -g$$

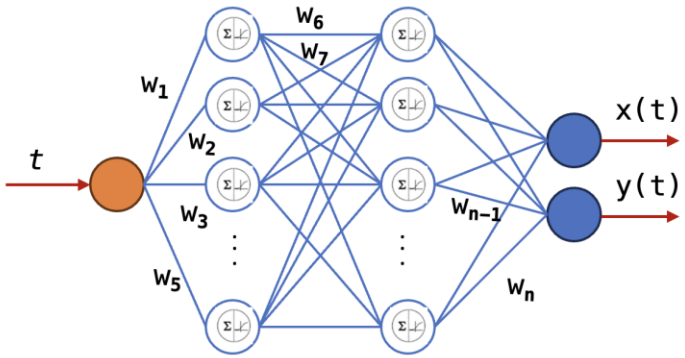
$$x(0) = x_0, \qquad y(0) = y_0$$

$$\frac{dx}{dt}(0) = v_0\cos\theta$$

$$\frac{dy}{dt}(0) = v_0\sin\theta$$







$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math

# ==============================
# 1. Configuration
# ==============================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)

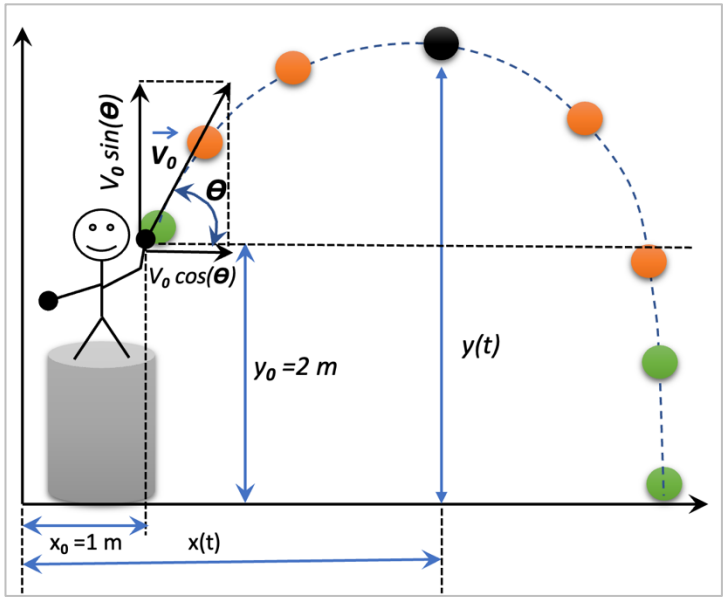
# Paramètres physiques
g = 9.81

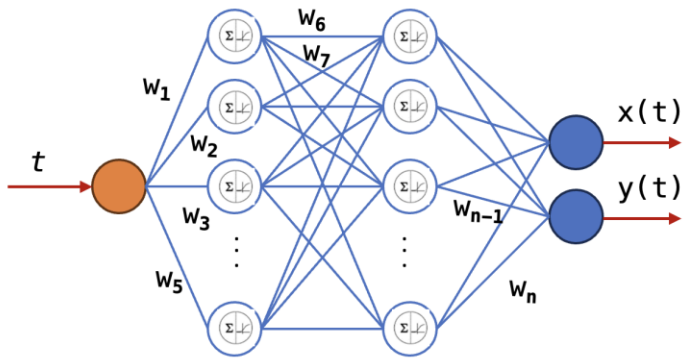
# Conditions initiales
x0 = 1.0
y0 = 2.0

teta = math.pi/3
v0 = 20 # m/s
vx0 = v0*math.cos(teta)
vy0 = v0*math.sin(teta)

# Temps de vol théorique pour y = 2
T_final = 2 * vy0 / g

print("Device :", device)
print("Temps de vol théorique :", T_final)
```



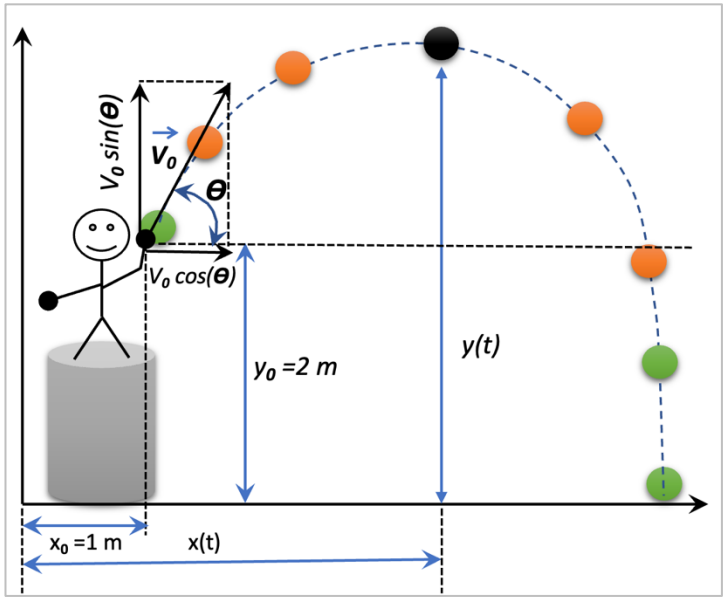


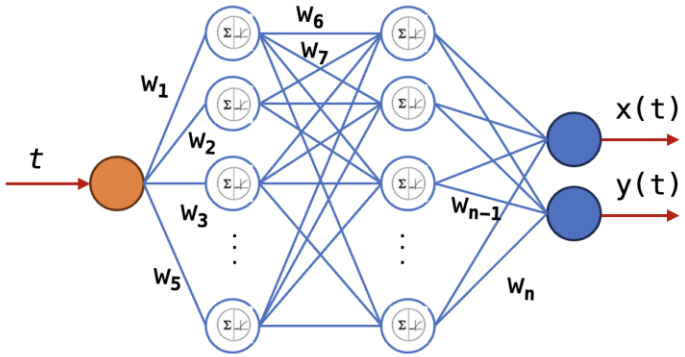
$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# ==============================
# 2. Helper pour les dérivées
# ==============================

def derivative(y, t):
    """
    Calcule dy/dt avec autograd.
    """
    return torch.autograd.grad(
        y,
        t,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
model = nn.Sequential(
        nn.Linear(1, 64),
        nn.Tanh(),

        nn.Linear(64, 64),
        nn.Tanh(),

        nn.Linear(64, 64),
        nn.Tanh(),

        nn.Linear(64, 2)
    ).to(device)
```

```
# ============================
# 3. Réseau PINN
# ============================

class ProjectilePINN(nn.Module):
    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(1, 64),
            nn.Tanh(),

            nn.Linear(64, 64),
            nn.Tanh(),

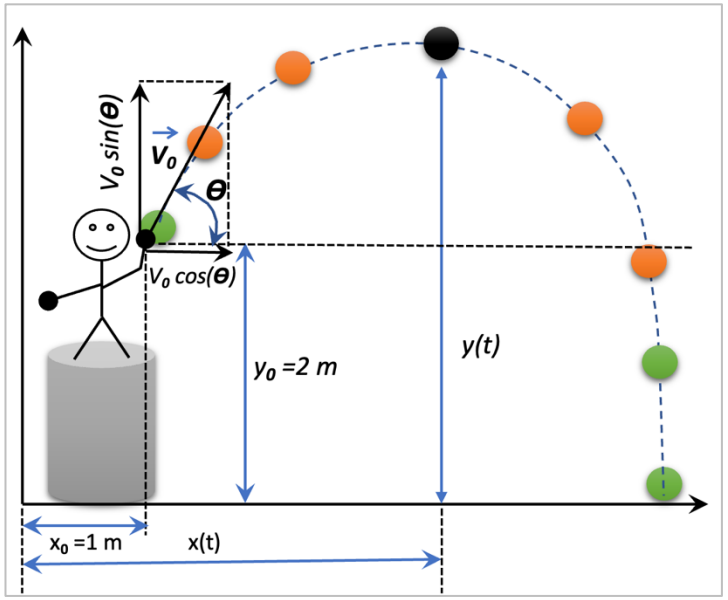
            nn.Linear(64, 64),
            nn.Tanh(),

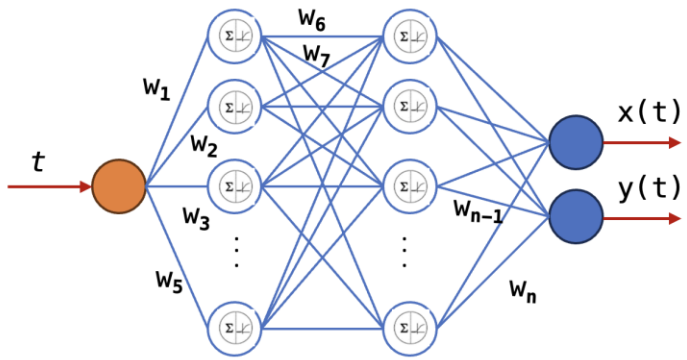
            nn.Linear(64, 64),
            nn.Tanh(),

            nn.Linear(64, 2)
        )

    def forward(self, t):
        return self.net(t)

model = ProjectilePINN().to(device)
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

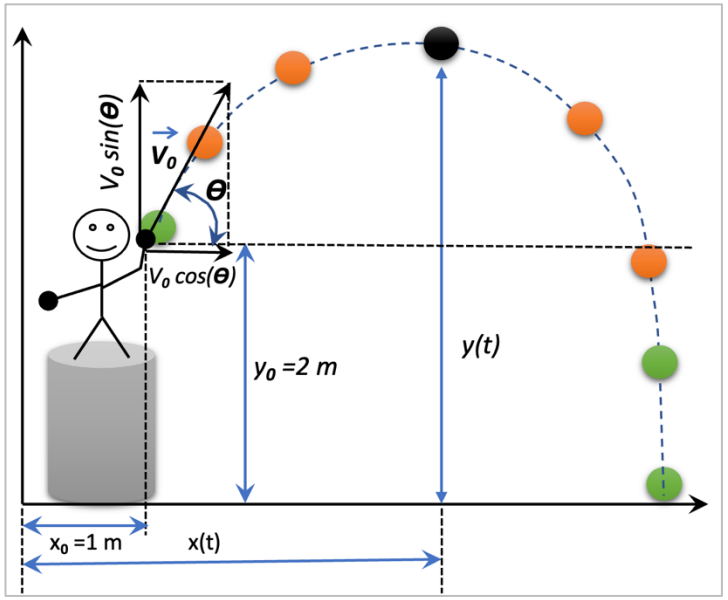
```
# ==============================
# 4. Points de collocation physique
# ==============================

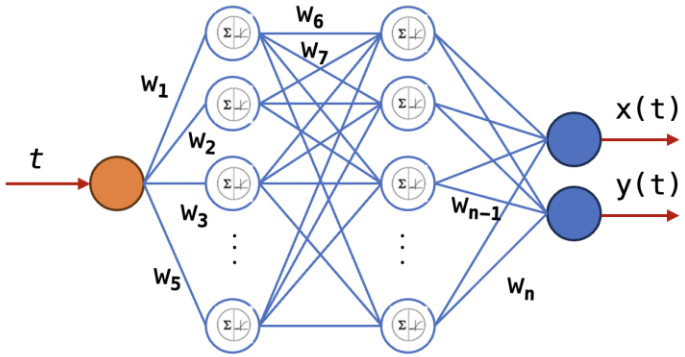
N_f = 200

t_f = torch.linspace(0, T_final, N_f).view(-1, 1).to(device)
t_f.requires_grad_(True)

# ==============================
# 5. Condition initiale t = 0
# ==============================

t0 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
t0.requires_grad_(True)
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# ==============================
# 6. Données expérimentales simulées
# ==============================

N_data = 10
noise_level = 0.9

t_data = torch.linspace(0, T_final, N_data).view(-1, 1).to(device)

# Solution théorique aux temps expérimentaux
x_data_exact = x0 + vx0 * t_data
y_data_exact = y0 + vy0 * t_data - 0.5 * g * t_data ** 2

# Bruit gaussien
noise_x = noise_level * torch.randn_like(x_data_exact)
noise_y = noise_level * torch.randn_like(y_data_exact)

# Données expérimentales bruitées
x_data_exp = x_data_exact + noise_x
y_data_exp = y_data_exact + noise_y

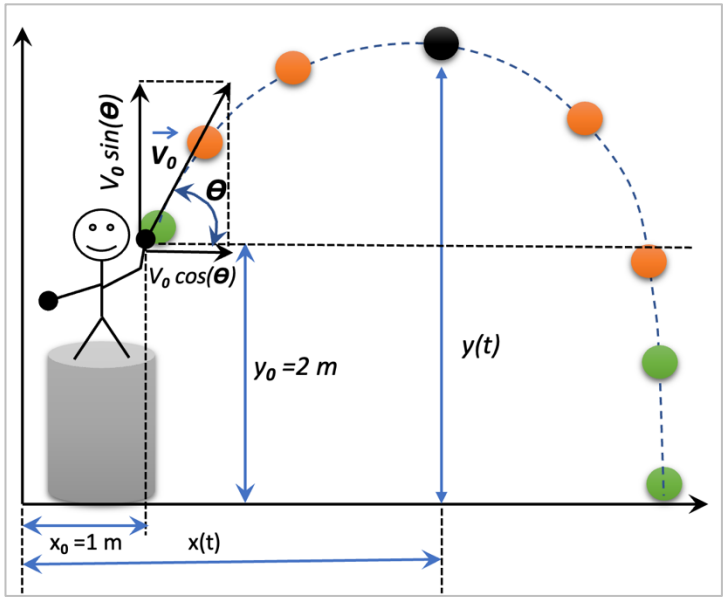
# Regrouper en un tenseur de shape : [N_data, 2]
data_exp = torch.cat([x_data_exp, y_data_exp], dim=1)
```

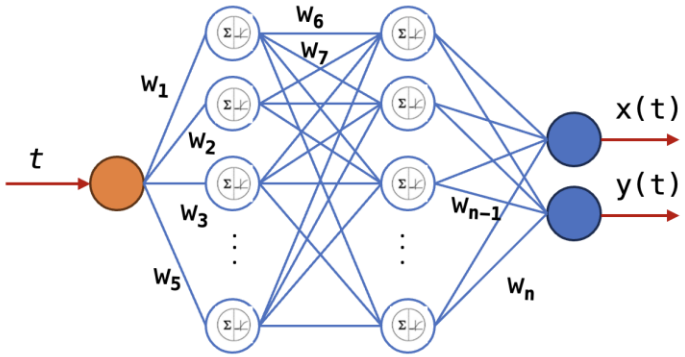
$$x(t) = x_0 + v_0 \cos(\theta)\, t$$

$$y(t) = y_0 + v_0 \sin(\theta)\, t - \frac{1}{2} g t^2$$

time
0.000000
0.588532
1.177065
1.765597
2.354130
2.942662
3.531194
4.119727
4.708259
5.296792

X	y
0.471534	2.219682
9.586019	15.523645
17.993155	25.836216
28.226082	33.179218
35.791260	35.383904
44.638611	36.379066
54.629513	32.692162
62.136314	26.421669
71.856407	15.405033
80.820435	2.086739






```
# ==============================
# 7. Optimiseur
# ==============================

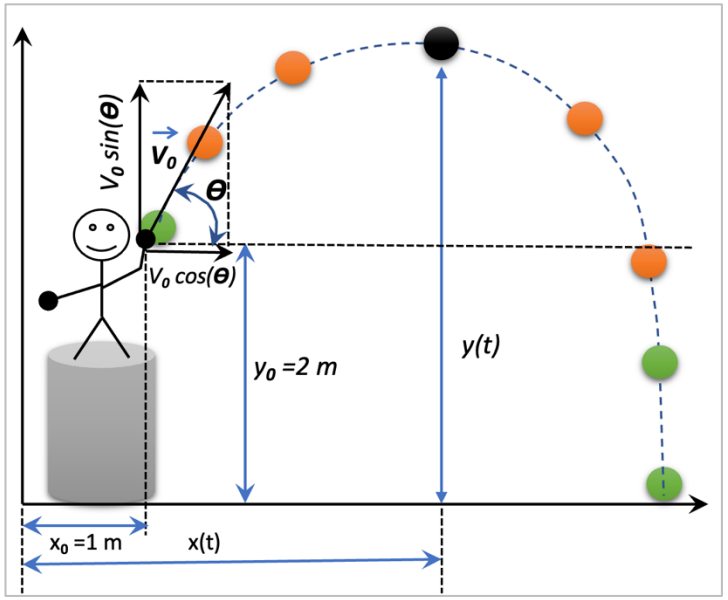
optimizer = optim.Adam(model.parameters(), lr=0.001)

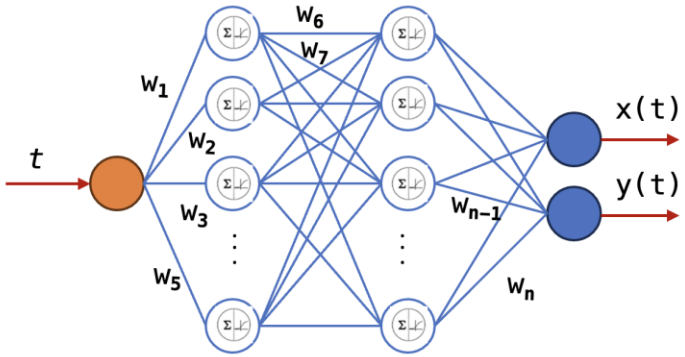
# Poids des différentes losses
lambda_physics = 1
lambda_ic = 1
lambda_data = 300
```

$$\mathcal{L}_{PINN} = \lambda_{data}\mathcal{L}_{data} + \lambda_{physics}\mathcal{L}_{physics} + \lambda_{IC}\mathcal{L}_{IC}$$

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# ============================
# 8. Entraînement
# ============================
epochs = 8000
for epoch in range(epochs):
    model.train()
    # -------------------------
    # Loss physique
    # -------------------------
    pred_f = model(t_f)
    x_pred = pred_f[:, 0:1]
    y_pred = pred_f[:, 1:2]

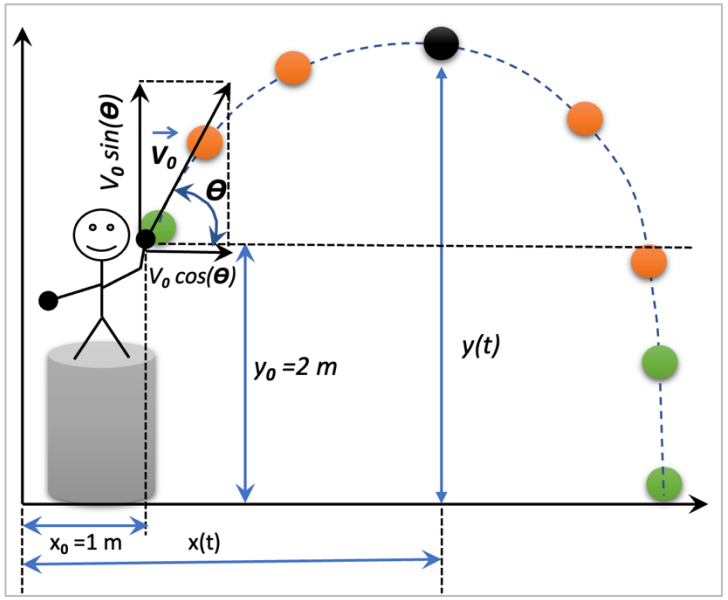
    # Dérivées premières
    x_t = derivative(x_pred, t_f)
    y_t = derivative(y_pred, t_f)

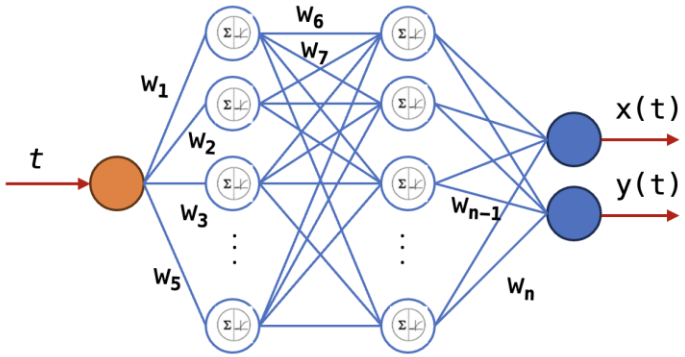
    # Dérivées secondes
    x_tt = derivative(x_t, t_f)
    y_tt = derivative(y_t, t_f)

    # Equations physiques :
    # x''(t) = 0
    # y''(t) = -g  => y''(t) + g = 0
    residual_x = x_tt
    residual_y = y_tt + g

    loss_physics = torch.mean(residual_x ** 2) + torch.mean(residual_y ** 2)
```

$$\mathcal{L}_{physics} = \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\left(\frac{d^2\hat{x}(t_i)}{dt^2}\right)^2 + \left(\frac{d^2\hat{y}(t_i)}{dt^2} + g\right)^2\right]$$





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# -------------------------
# Loss conditions initiales
# -------------------------

pred0 = model(t0)

x0_pred = pred0[:, 0:1]
y0_pred = pred0[:, 1:2]

x0_t = derivative(x0_pred, t0)
y0_t = derivative(y0_pred, t0)

loss_ic_position = (x0_pred - x0) ** 2 + (y0_pred - y0) ** 2
loss_ic_velocity = (x0_t - vx0) ** 2 + (y0_t - vy0) ** 2

loss_ic = torch.mean(loss_ic_position + loss_ic_velocity)

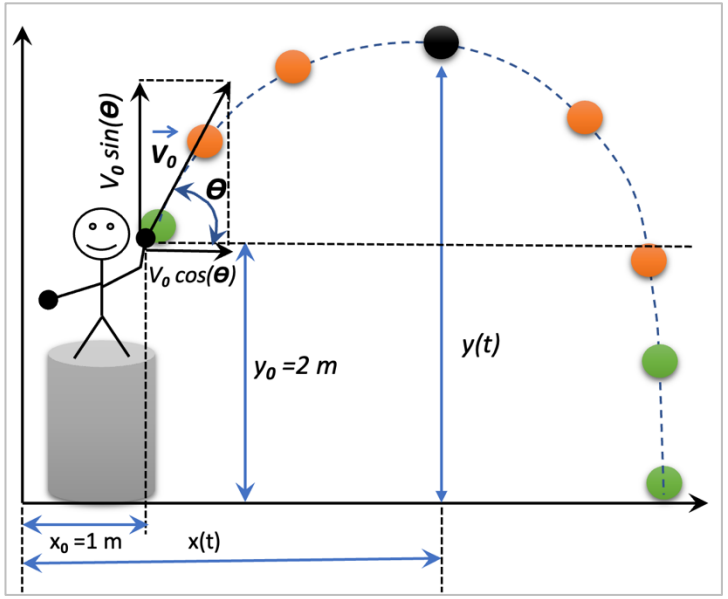
# -------------------------
# Loss données expérimentales
# -------------------------

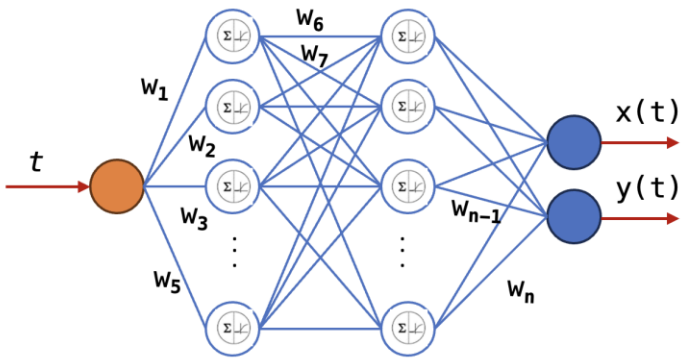
pred_data = model(t_data)

loss_data = torch.mean((pred_data - data_exp) ** 2)
```

$$\mathcal{L}_{IC} = (\hat{x}(0) - x_0)^2 + (\hat{y}(0) - y_0)^2 + \left(\frac{d\hat{x}}{dt}(0) - v_0\cos\theta\right)^2 + \left(\frac{d\hat{y}}{dt}(0) - v_0\sin\theta\right)^2$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$






```
# -------------------------
  # Loss totale
  # -------------------------

  loss = (
      lambda_physics * loss_physics
      + lambda_ic * loss_ic
      + lambda_data * loss_data
  )

  # -------------------------
  # Backpropagation
  # -------------------------

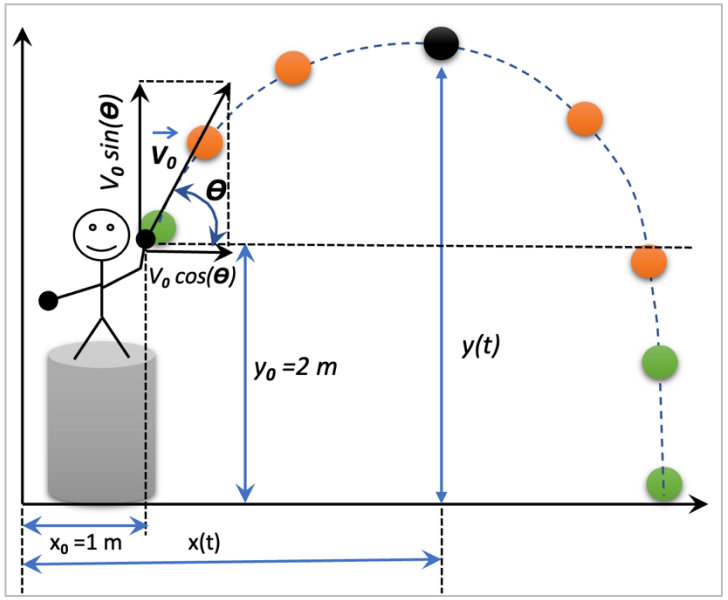
  optimizer.zero_grad()
  loss.backward()
  optimizer.step()

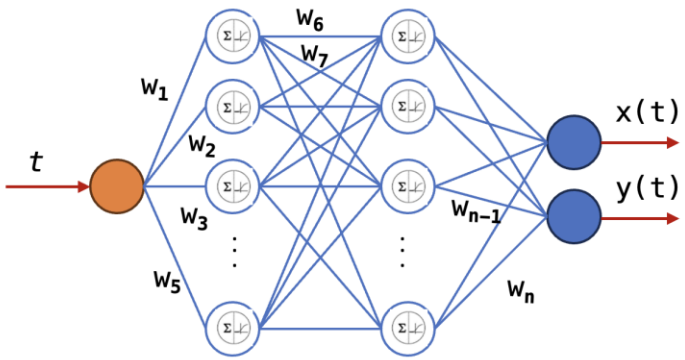
  if (epoch + 1) % 1000 == 0:
      print(
          f"Epoch [{epoch+1}/{epochs}] "
          f"Loss: {loss.item():.6f} | "
          f"Physics: {loss_physics.item():.6f} | "
          f"IC: {loss_ic.item():.6f} | "
          f"Data: {loss_data.item():.6f}"
      )
```

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

$$\mathcal{L}_{PINN} = \lambda_{data}\mathcal{L}_{data} + \lambda_{physics}\mathcal{L}_{physics} + \lambda_{IC}\mathcal{L}_{IC}$$





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# ============================
# 9. Évaluation
# ============================

model.eval()

t_test = torch.linspace(0, T_final, 300).view(-1, 1).to(device)

with torch.no_grad():
    pred_test = model(t_test)

x_pinn = pred_test[:, 0:1]
y_pinn = pred_test[:, 1:2]

# Solution théorique exacte
x_exact = x0 + vx0 * t_test
y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2

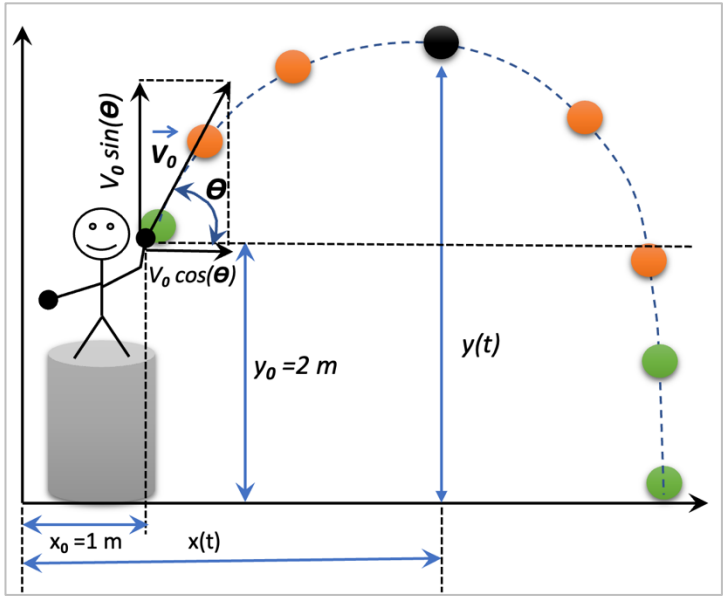
# ============================
# 10. Conversion CPU/Numpy
# ============================

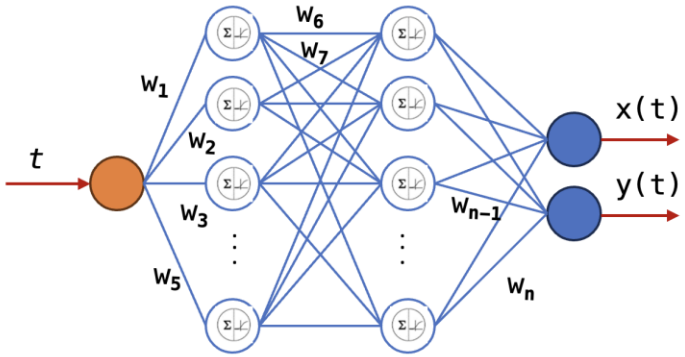
t_np = t_test.cpu().numpy()

x_pinn_np = x_pinn.cpu().numpy()
y_pinn_np = y_pinn.cpu().numpy()

x_exact_np = x_exact.cpu().numpy()
y_exact_np = y_exact.cpu().numpy()

t_data_np = t_data.cpu().numpy()
x_data_exp_np = x_data_exp.cpu().numpy()
y_data_exp_np = y_data_exp.cpu().numpy()
```





$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

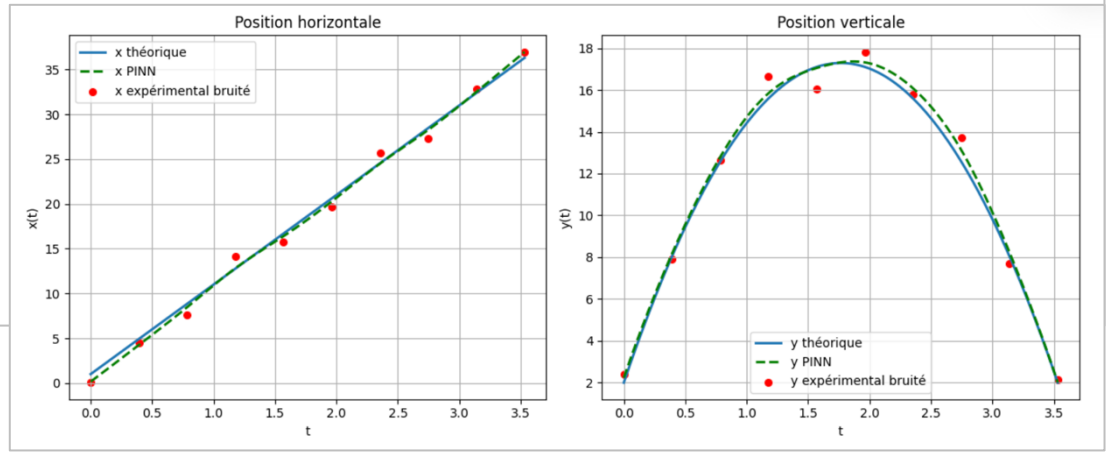
$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

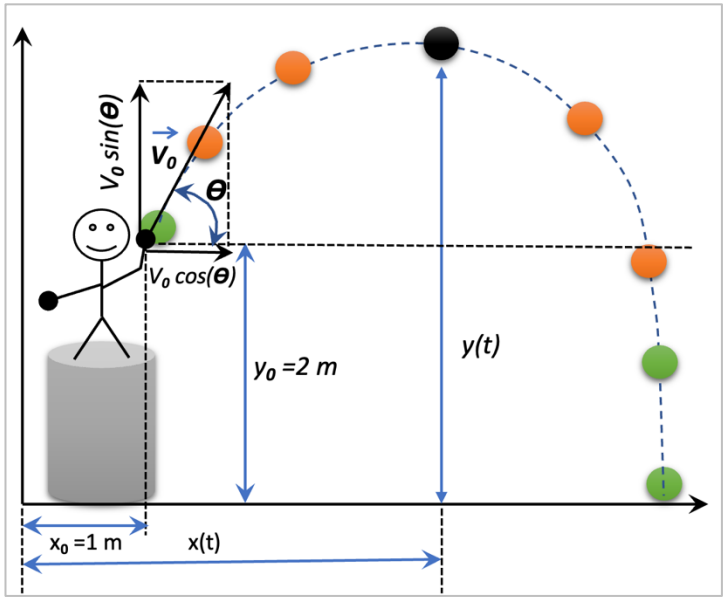
```
# ============================
# 11. Visualisation x(t) et y(t)
# ============================

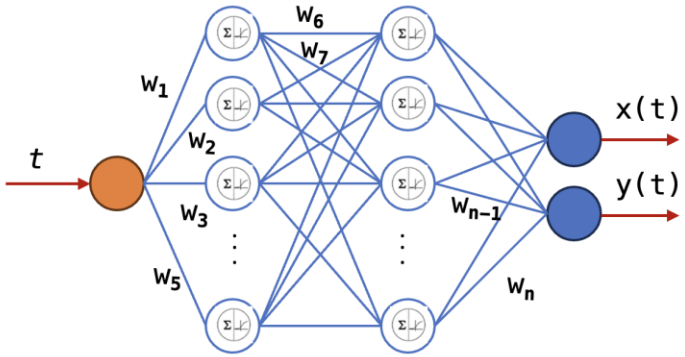
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(t_np, x_exact_np, label="x théorique", linewidth=2)
plt.plot(t_np, x_pinn_np, "--",color="green", label="x PINN", linewidth=2)
plt.scatter(t_data_np, x_data_exp_np, color="red", s=30, label="x expérimental bruité")
plt.xlabel("t")
plt.ylabel("x(t)")
plt.title("Position horizontale")
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(t_np, y_exact_np, label="y théorique", linewidth=2)
plt.plot(t_np, y_pinn_np, "--", color="green", label="y PINN", linewidth=2)
plt.scatter(t_data_np, y_data_exp_np, color="red",s=30,label="y expérimental bruité")
plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("Position verticale")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```







$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i)^2 + (\hat{y}(t_i) - y_i)^2\right]$$

```
# ============================
# 12. Visualisation trajectoire y(x)
# ============================

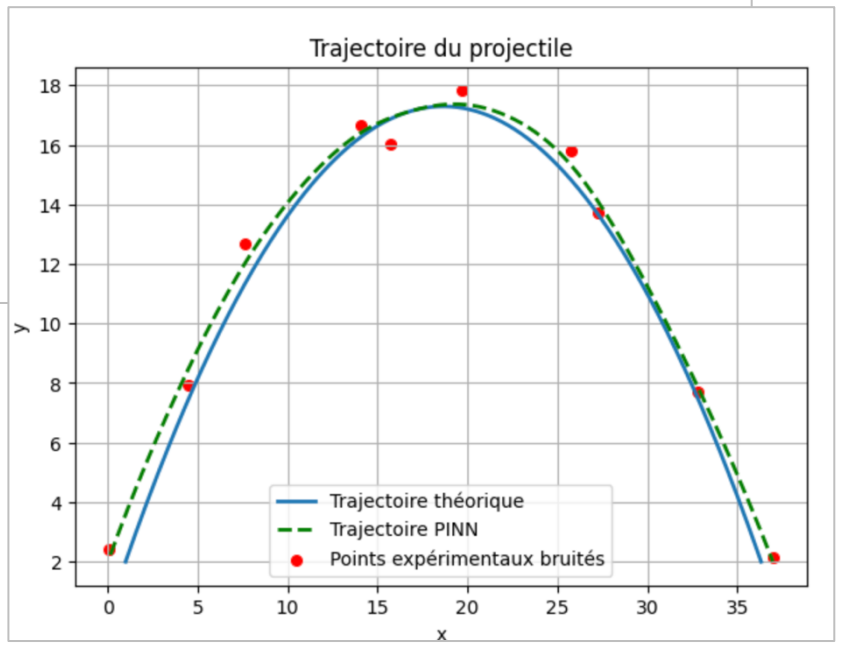
plt.figure(figsize=(7, 5))

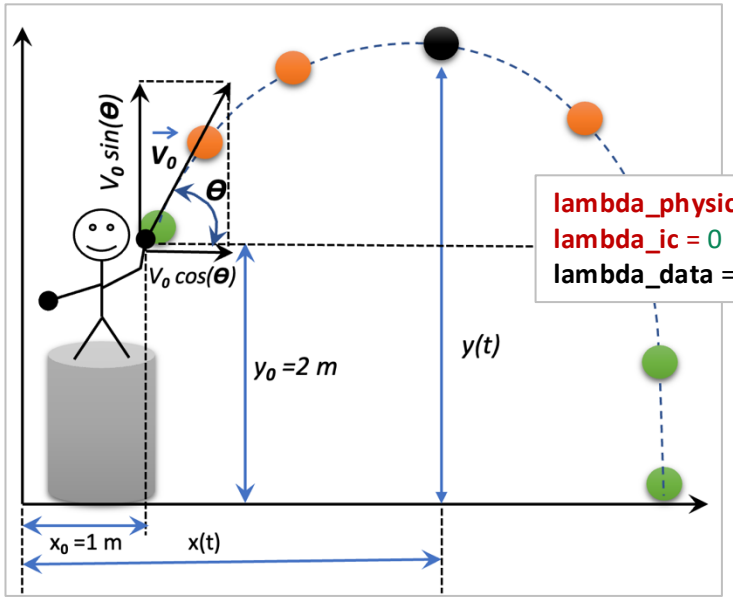
plt.plot( x_exact_np, y_exact_np, label="Trajectoire théorique",linewidth=2)

plt.plot(x_pinn_np, y_pinn_np, "--", color="green", label="Trajectoire PINN",
   linewidth=2
)

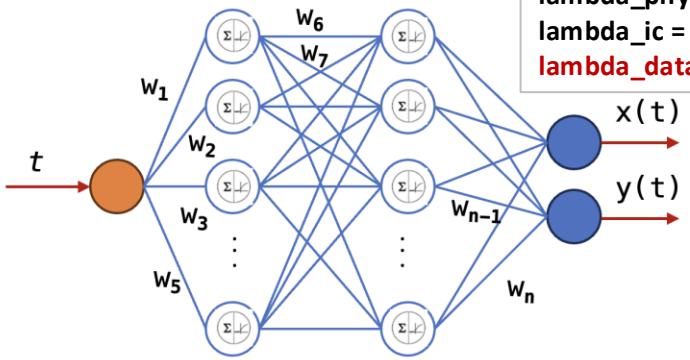
plt.scatter( x_data_exp_np, y_data_exp_np, color="red", s=30,
   label="Points expérimentaux bruités"
)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectoire du projectile")
plt.legend()
plt.grid()
plt.show()
```





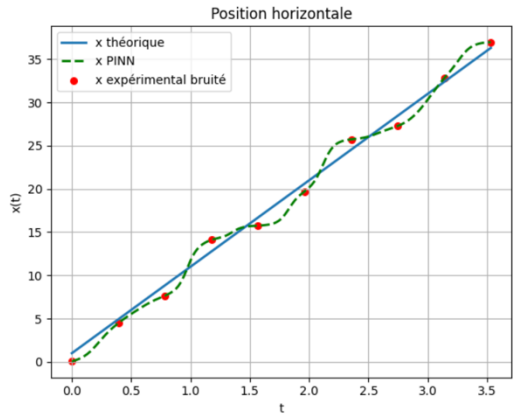
lambda_physics = 0
lambda_ic = 0
lambda_data = 1

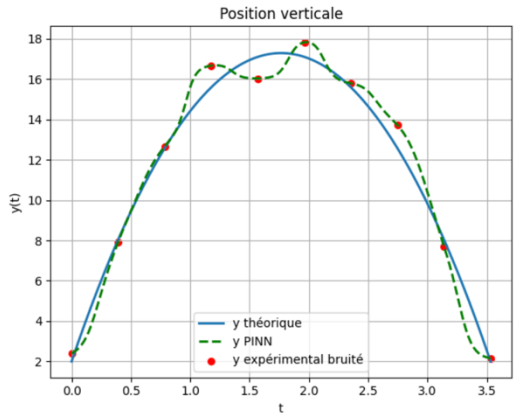


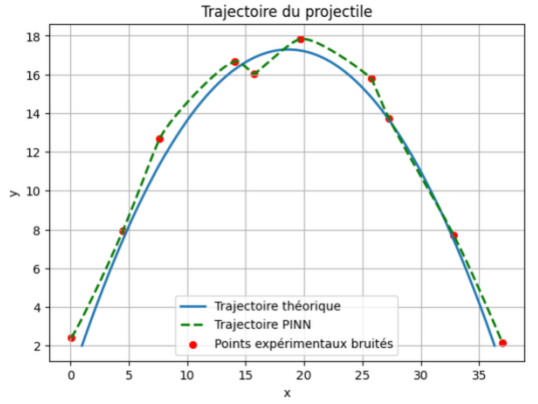
lambda_physics = 1
lambda_ic = 1
lambda_data = 0

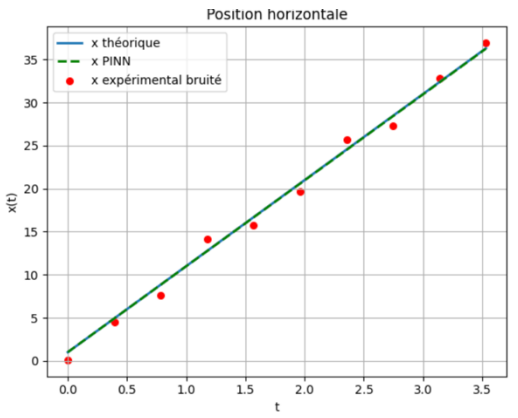
$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

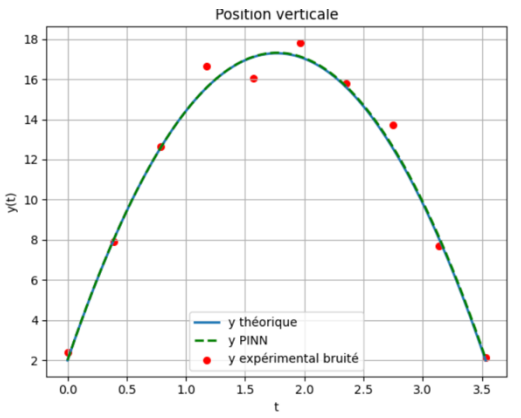
lambda_physics = 1
lambda_ic = 1
lambda_data = 1

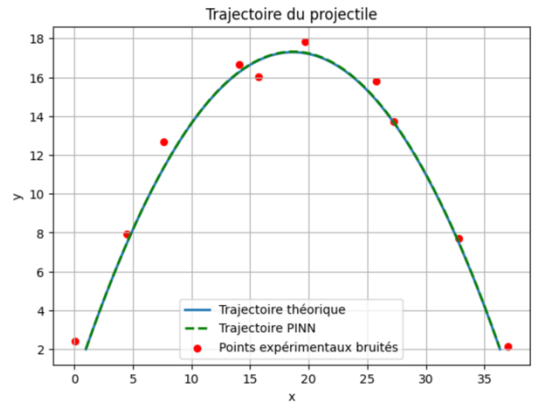


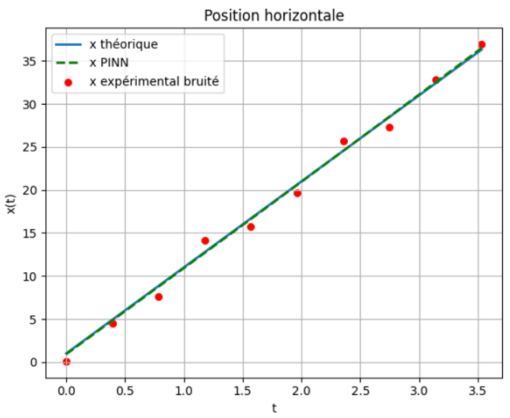


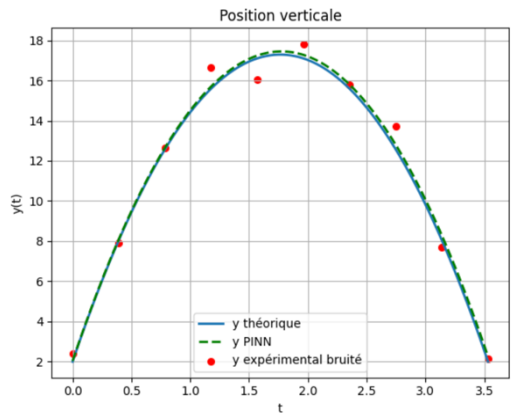


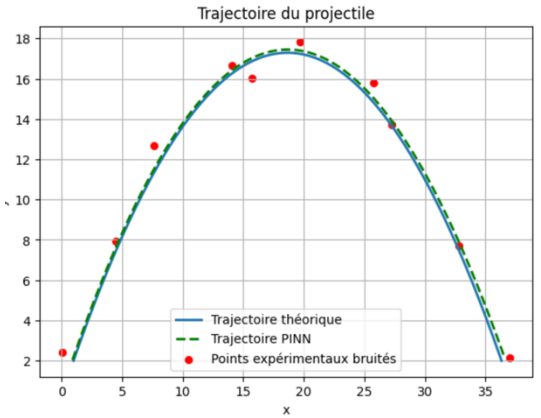










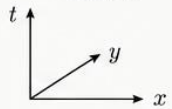


PINN pour l'équation de la chaleur 2D

Entrées (espace–temps)

Coordonnées : (x, y, t)

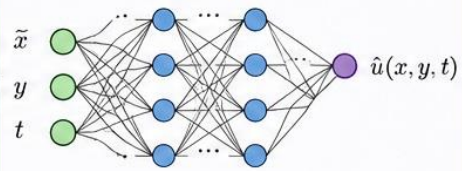
$x \in [0, L_x]$
$y \in [0, L_y]$
$t \in [0, T]$



Ex : $t \to (x, y, t)$

Réseau de neurones (approximateur)

$$u_\theta(x, y, t) \approx u(x, y, t)$$



Paramètres : θ (poids et biais)

Modèle physique : équation de la chaleur 2D

Équation différentielle (conduction thermique)

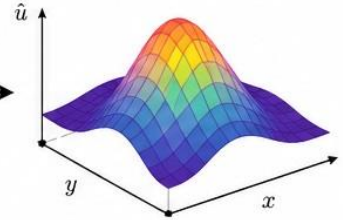
$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad \text{dans} \quad \Omega \times (0, T]$$

avec α : diffusivité thermique (constante)

$\Omega = [0, L_x] \times [0, L_y]$: domaine spatial

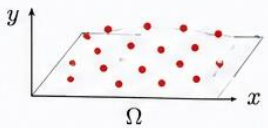
Solution prédite $\hat{u}(x, y, t)$

(à un instant t fixé)



1) Perte de données (Data loss)

Sur des mesures (x_i, y_i, t_i)

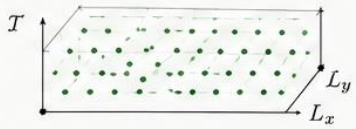


- Données mesurées
- Prédiction PINN

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} |\hat{u}(x_i, y_i, t_i) - u_i^{\text{obs}}|^2$$

2) Perte physique (Physics loss)

Sur des points de collocation $(x_f, y_f, t_f) \in \Omega \times (0, T]$



Résidu de l'équation de la chaleur :

$$f(x, y, t) = \frac{\partial \hat{u}}{\partial t} - \alpha \left(\frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \right)$$

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_f} \sum_{j=1}^{N_f} |f(x_f, y_f, t_f)|^2$$

3) Perte conditions initiales (IC loss)

À $t = 0$, $\quad u(x, y, 0) = u_0(x, y)$

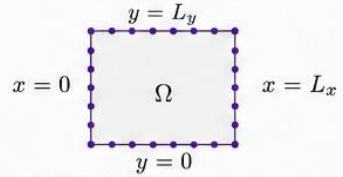
Sur des points $(x_{ic}, y_{ic}) \in \Omega$



$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}} \sum_{k=1}^{N_{\text{ic}}} |\hat{u}(x_{\text{ic}}, y_{\text{ic}}, 0) - u_0(x_{\text{ic}}, y_{\text{ic}})|^2$$

4) Perte conditions aux limites (BC loss)

Sur la frontière $\partial\Omega$ et pour $t \in (0, T]$



- Points de bord

(Dirichlet : $u = g_D$ ou Neumann : $\partial u / \partial n = g_N$)

$$\mathcal{L}_{\text{bc}} = \frac{1}{N_{\text{bc}}} \sum_{m=1}^{N_{\text{bc}}} |\mathcal{B}[\hat{u}](x_b, y_b, t_b) - g_b|^2$$

Perte totale pondérée

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}$$

Minimiser $\mathcal{L}$ w.r.t. θ (paramètres du réseau)

Backpropagation (descente de gradient)

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Mise à jour des paramètres

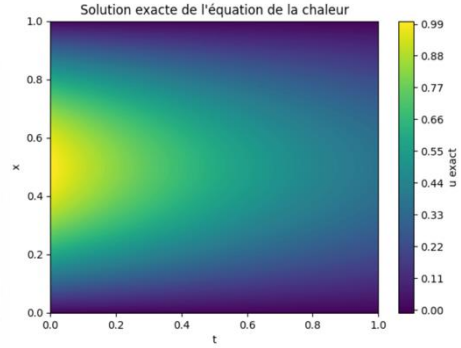
θ (poids et biais)

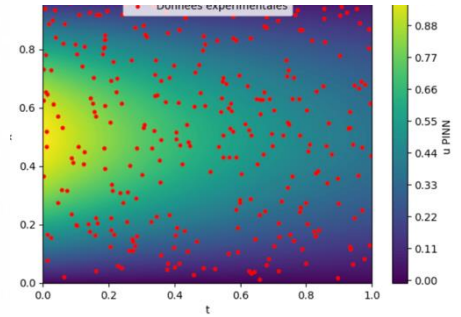
Arrêt si

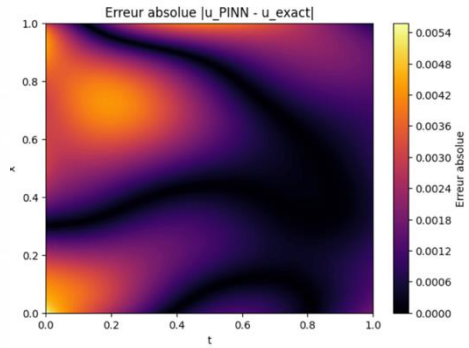
- $\mathcal{L} < \epsilon$
- ou nombre max d'itérations atteint

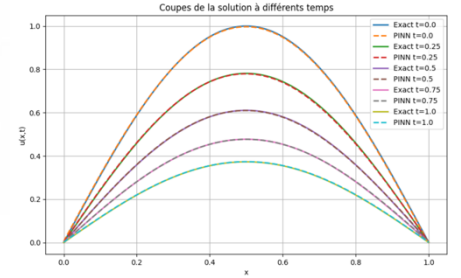
Légende

N_d : nb. de points de données
N_f : nb. de points de collocation
N_{ic} : nb. de points IC
N_{bc} : nb. de points BC
α : diffusivité thermique
$\mathcal{B}[\hat{u}]$: opérateur de bord (Dirichlet ou Neumann)









PINN pour un circuit électrique RLC série

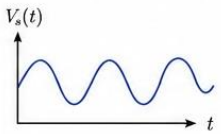
Réseau de neurones informé par la physique

1) Entrées (espace-temps)

- t : temps

Paramètres du circuit

R, L, C, V_s

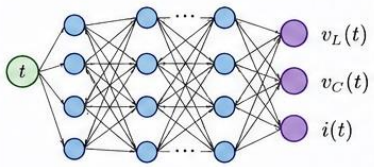


Ex: $V_s(t) = V_m \sin(\omega t)$

2) Réseau de neurones (approximateur)

Réseau de neurones $\mathcal{N}_\theta$

Entrée : t Sorties :

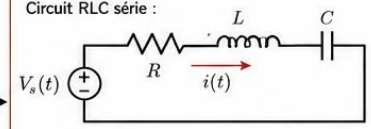


Paramètres : θ (poids et biais)

3) Modèle physique (lois du circuit)

Équations différentielles (lois de Kirchhoff)

Circuit RLC série :



Équation (KVL) :

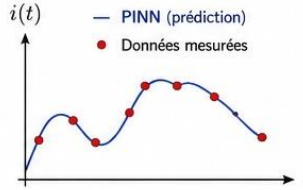
$$v_R(t) + v_L(t) + v_C(t) = V_s(t)$$

avec $\begin{cases} v_R(t) = R\ i(t) \\ v_L(t) = L\ \dfrac{di(t)}{dt} \\ i(t) = C\ \dfrac{dv_C(t)}{dt} \end{cases}$

Résidus (physique)

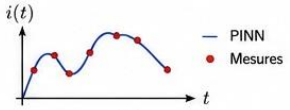
$$f_1(t) = v_R + v_L + v_C - V_s(t)$$
$$f_2(t) = v_L - L\ \frac{di}{dt}$$
$$f_3(t) = i - C\ \frac{dv_C}{dt}$$

Sorties du PINN vs données (si disponibles)



4) Perte de données (Data loss)

Sur les mesures disponibles



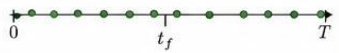
$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{k=1}^{N_d} \left\| \hat{y}(t_k) - y_k^{\text{obs}} \right\|^2$$

avec $\hat{y} = [i,\ v_C,\ v_L]^{\mathrm{T}}$

y^{obs} : données mesurées

5) Perte physique (Physics loss)

Sur des points de collocation t_f dans $[0, T]$



$$\mathcal{L}_{\text{phys}} = \frac{1}{N_f} \sum_{j=1}^{N_f} \left(\omega_1 f_1(t_j)^2 + \omega_2 f_2(t_j)^2 + \omega_3 f_3(t_j)^2 \right)$$

où f_1, f_2, f_3 : résidus ci-dessus

(dérivées calculées par autodiff)

6) Perte conditions initiales (IC loss)

À $t = 0$

Conditions initiales :

$$i(0) = i_0$$
$$v_C(0) = v_{C0}$$
$$v_L(0) = v_{L0} = L\ \frac{di}{dt}(0)$$

$$\mathcal{L}_{\text{ic}} = \left\| i(0) - i_0 \right\|^2 + \left\| v_C(0) - v_{C0} \right\|^2 + \left\| v_L(0) - v_{L0} \right\|^2$$

7) Perte conditions aux limites (BC loss)

Bornes temporelles $t = 0$ et $t = T$

(ou périodicité pour régime permanent)

Ex: périodicité (régime sinusoïdal établi)

$$y(0) = y(T)$$
$$\frac{dy}{dt}(0) = \frac{dy}{dt}(T)$$

$$\mathcal{L}_{\text{bc}} = \left\| y(0) - y(T) \right\|^2 + \left\| \frac{dy}{dt}(0) - \frac{dy}{dt}(T) \right\|^2$$

8) Perte totale pondérée

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}$$

9) Minimiser $\mathcal{L}$ par rapport à θ

Backpropagation (descente de gradient)

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Mise à jour des paramètres

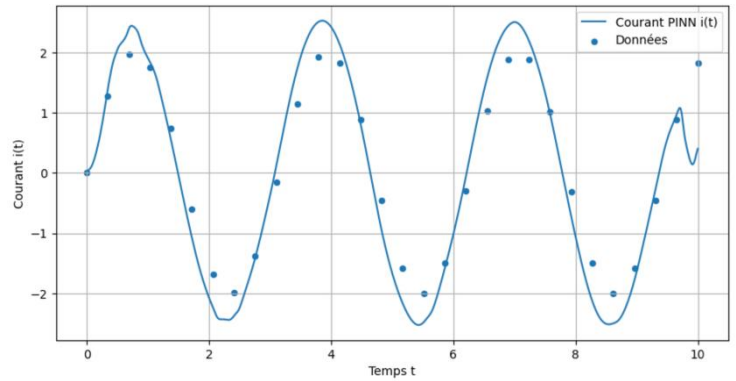
θ (poids et biais)

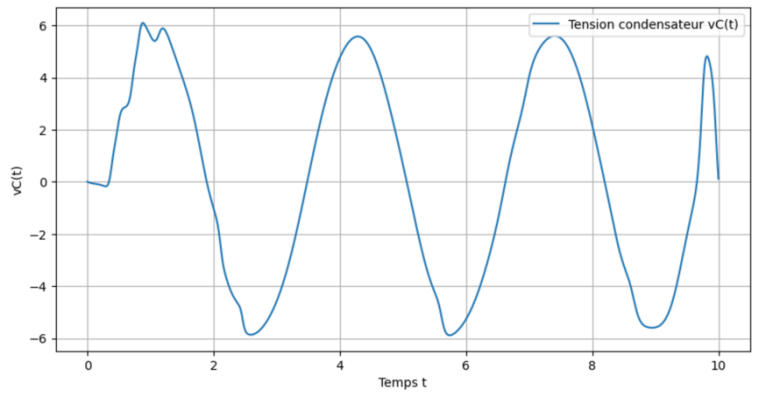
Arrêt si

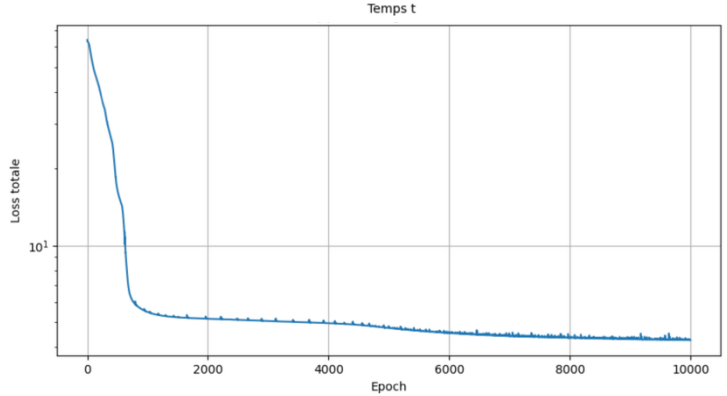
- $\mathcal{L} < \varepsilon$
- ou nombre max d'itérations atteint

Légende

- R : résistance (Ω)
- L : inductance (H)
- C : capacité (F)
- $V_s(t)$: source de tension
- $i(t)$: courant
- v_R, v_L, v_C : tensions aux bornes
- t_f : points de collocation
- N_d, N_f : nombres de points







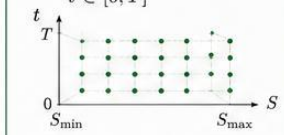
PINN pour la valorisation d'options Européennes (équation de Black–Scholes)

On apprend le prix de l'option $u(S,t)$ à partir des données du marché et des lois de la physique (absence d'arbitrage)

Entrées (espace–temps)

Variables :
S : prix de l'actif sous-jacent
t : temps (maturité restante)

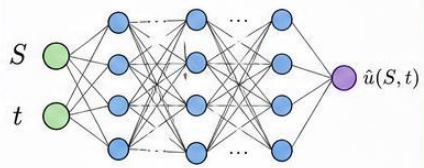
$S \in [S_{\min}, S_{\max}]$
$t \in [0, T]$



Entrée du réseau : (S,t)

Réseau de neurones (approximateur)

$u_\theta(S,t) \approx u(S,t)$: prix de l'option



Paramètres : θ (poids et biais)

Modèle physique : équation de Black–Scholes

EDP sous la mesure risque-neutre (r : taux sans risque, σ : volatilité)

$$\frac{\partial u}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 u}{\partial S^2} + rS\frac{\partial u}{\partial S} - ru = 0$$

dans $\Omega = (S_{\min}, S_{\max}) \times (0, T)$

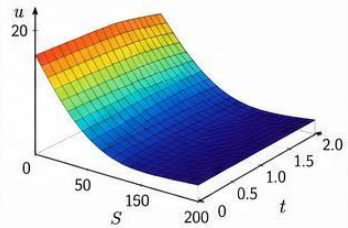
où $u(S,t)$ = prix théorique de l'option

Résidu (physique)

$$f(S,t;\theta) = \frac{\partial \hat{u}}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 \hat{u}}{\partial S^2} + rS\frac{\partial \hat{u}}{\partial S} - r\hat{u}$$

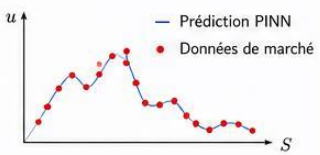
Solution prédite $\hat{u}(S,t)$

Surface $u(S,t)$ à un instant t fixé



1) Perte de données (Data loss)

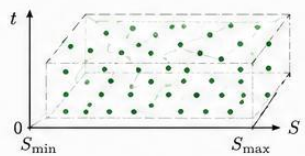
Sur les prix de marché observés



$$\mathcal{L}_{\text{data}} = \frac{1}{N_d}\sum_{i=1}^{N_d} |\hat{u}(S_i,t_i) - u_i^{mkt}|^2$$

2) Perte physique (Physics loss)

Sur des points de collocation (S_j, t_j) dans Ω

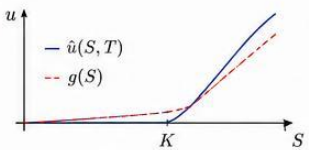


$$\mathcal{L}_{\text{phys}} = \frac{1}{N_f}\sum_{j=1}^{N_f} |f(S_j,t_j;\theta)|^2$$

3) Perte condition terminale (IC loss)

À l'échéance $t = T$, payoff connu

Option Call Européenne : $g(S) = \max(S-K, 0)$



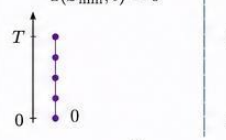
$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}}\sum_{k=1}^{N_{\text{ic}}} |\hat{u}(S_k,T) - g(S_k)|^2$$

4) Perte conditions aux limites (BC loss)

Sur les frontières de Ω

$S = S_{\min}$ (Dirichlet) $\quad u(S_{\min}, t) = 0$

$S = S_{\max}$ (Dirichlet asymptotique) $\quad u(S_{\max}, t) = S_{\max} - Ke^{-rt}$



$$\mathcal{L}_{\text{bc},1} = \frac{1}{N_{\text{bc},1}}\sum_{m=1}^{N_{\text{bc},1}} |\hat{u}(S_{\min},t_m)|^2 \qquad \sum_{m=1}^{N_{\text{bc},2}} |\hat{u}(S_{\max},t_m) - g_{\max}(t_m)|^2$$

5) Perte totale pondérée

$$\mathcal{L} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}} + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}} + \lambda_{\text{bc1}}\mathcal{L}_{\text{bc},1} + \lambda_{\text{bc2}}\mathcal{L}_{\text{bc},2}$$

Légende

- N_d : nb. de données de marché
- N_f : nb. de points de collocation
- N_{ic} : nb. de points IC (à $t = T$)
- $N_{\text{bc},1}$, $N_{\text{bc},2}$: nb. de points BC
- λ_+ : poids (hyperparamètres)

6) Minimiser $\mathcal{L}$ w.r.t. θ

Backpropagation (descente de gradient)

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

7) Mise à jour des paramètres

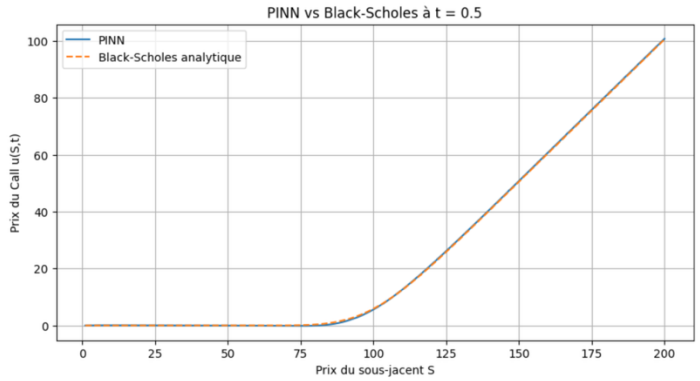
θ (poids et biais)

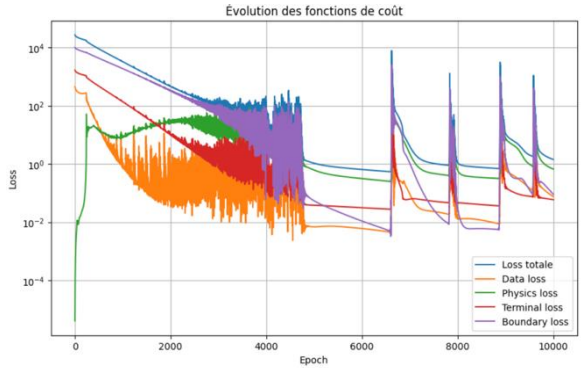
8) Arrêt si

- $\mathcal{L} < \epsilon$
- ou nombre max d'itérations atteint

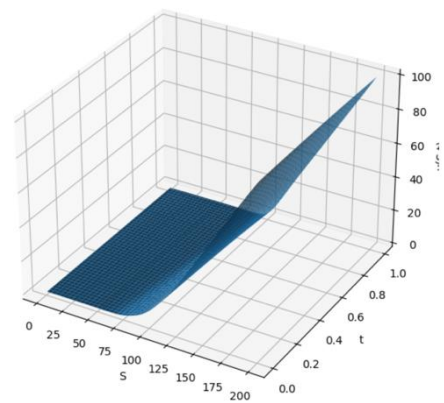
Exemple numérique

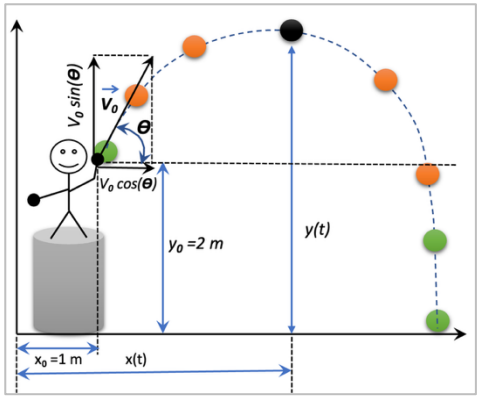
- Actif sans dividende
- $r = 1\%$, $\sigma = 20\%$, $K = 100$, $T = 1$ an
- Données : prix d'options du marché
- Sortie : surface de prix $u(S,t)$

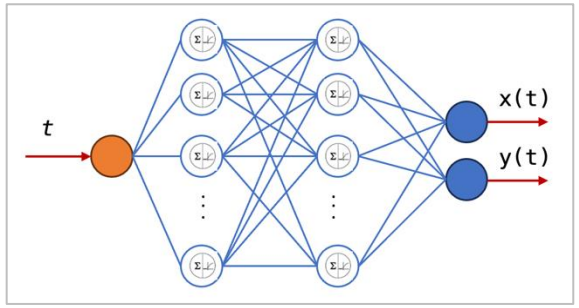




Surface prédite par le PINN








```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math

# Paramètres physiques
g = 9.81

# Conditions initiales
x0 = 1.0
y0 = 2.0

teta = math.pi/3
v0 = 20 # m/s
vx0 = v0*math.cos(teta)
vy0 = v0*math.sin(teta)

# Temps de vol théorique
# pour y = 0
T_final = 2 * vy0 / g
```

```
# ============================
# 2. Helper pour les dérivées
# ============================

def derivative(y, t):
    """
    Calcule dy/dt avec autograd.
    """
    return torch.autograd.grad(
    y,
    t,
    grad_outputs=torch.ones_like(y),
    create_graph=True
    )[0]

# ============================
# 3. Réseau PINN
# ============================

pinn_model = nn.Sequential(
        nn.Linear(1, 64),
        nn.Tanh(),
        nn.Linear(64, 64),
        nn.Tanh(),
        nn.Linear(64, 2)
)
```

```
# ============================
# 4. Points de collocation physique
# ============================

N_f = 200

t_f = torch.linspace(0, T_final, N_f).view(-1, 1)
t_f.requires_grad_(True)

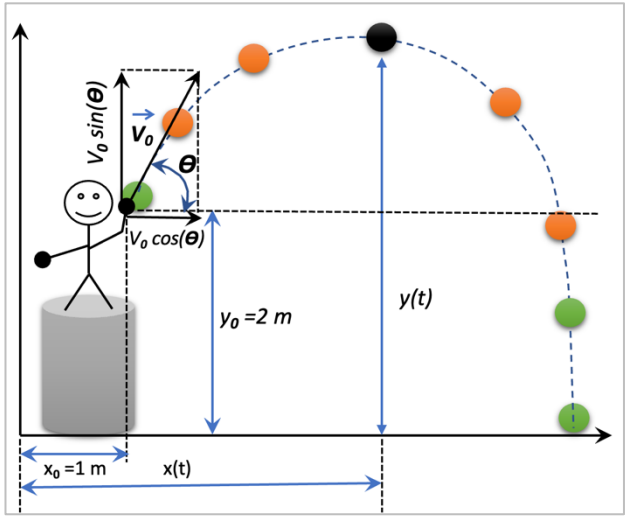
# ============================
# 5. Condition initiale t = 0
# ============================

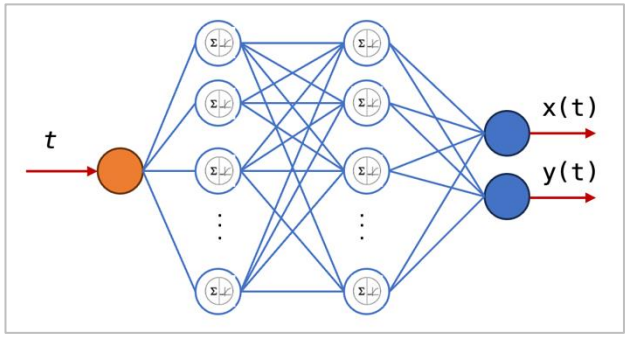
t0 = torch.tensor([[0.0]], dtype=torch.float32)
t0.requires_grad_(True)

# ============================
# 7. Optimiseur
# ============================

optimizer = optim.Adam(pinn_model.parameters(), lr=0.001)

# ============================
# 8. Entraînement
# ============================
```





$$\frac{d^2x}{dt^2} = 0$$

$$\frac{d^2y}{dt^2} = -g$$

$$x(0) = x_0, \qquad y(0) = y_0$$

$$\frac{dx}{dt}(0) = v_0 \cos\theta$$

$$\frac{dy}{dt}(0) = v_0 \sin\theta$$

```
epochs = 8000
for epoch in range(epochs):
    pinn_model.train()
    # Loss physique
    pred_f = pinn_model(t_f)
    x_pred = pred_f[:, 0:1]
    y_pred = pred_f[:, 1:2]
    # Dérivées premières
    x_t = derivative(x_pred, t_f)
    y_t = derivative(y_pred, t_f)
    # Dérivées secondes
    x_tt = derivative(x_t, t_f)
    y_tt = derivative(y_t, t_f)
    # Equations physiques :
    # x''(t) = 0
    # y''(t) = -g  => y''(t) + g = 0
    residual_x = x_tt
    residual_y = y_tt + g
    loss_physics = torch.mean(residual_x ** 2) + torch.mean(residual_y ** 2)
    # Loss conditions initiales
    pred0 = pinn_model(t0)
    x0_pred = pred0[:, 0:1]
    y0_pred = pred0[:, 1:2]
    x0_t = derivative(x0_pred, t0)
    y0_t = derivative(y0_pred, t0)
    loss_ic_position = (x0_pred - x0) ** 2 + (y0_pred - y0) ** 2
    loss_ic_velocity = (x0_t - vx0) ** 2 + (y0_t - vy0) ** 2
    loss_ic = torch.mean(loss_ic_position + loss_ic_velocity)
    # Loss totale
    loss = loss_physics +loss_ic
    # Backpropagation
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 1000 == 0:
        print( f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.6f} |  » )
```

```
# ============================
# 9. Évaluation
# ============================

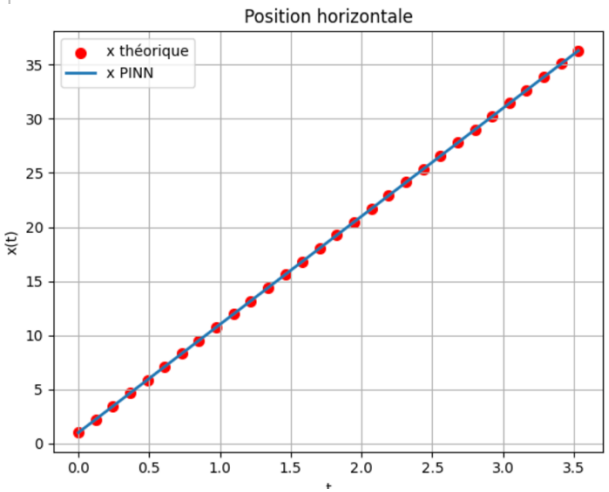
pinn_model.eval()

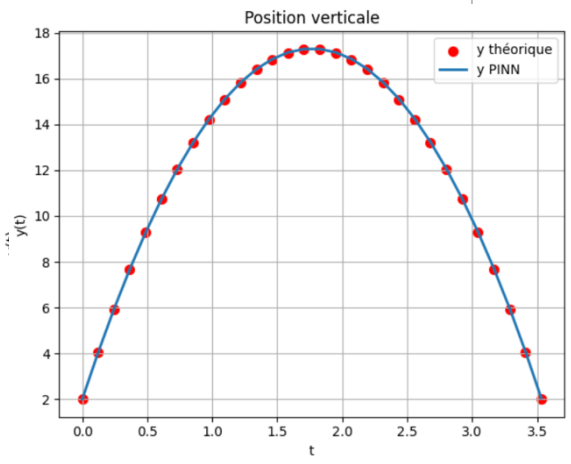
t_test = torch.linspace(0, T_final, 30).view(-1, 1)

with torch.no_grad():
    pred_test = pinn_model(t_test)

x_pinn = pred_test[:, 0:1]
y_pinn = pred_test[:, 1:2]

# Solution théorique exacte
x_exact = x0 + vx0 * t_test
y_exact = y0 + vy0 * t_test - 0.5 * g * t_test ** 2
```






```
# ============================
# 11. Visualisation x(t) et y(t)
# ============================

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(t_test, x_exact, label="x théorique", color="red",linewidth=2)
plt.plot(t_test, x_pinn,label="x PINN",  linewidth=2)

plt.xlabel("t")
plt.ylabel("x(t)")
plt.title("Position horizontale")
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.scatter(t_test, y_exact, label="y théorique", color="red", linewidth=2)
plt.plot(t_test, y_pinn, label="y PINN", linewidth=2)

plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("Position verticale")
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

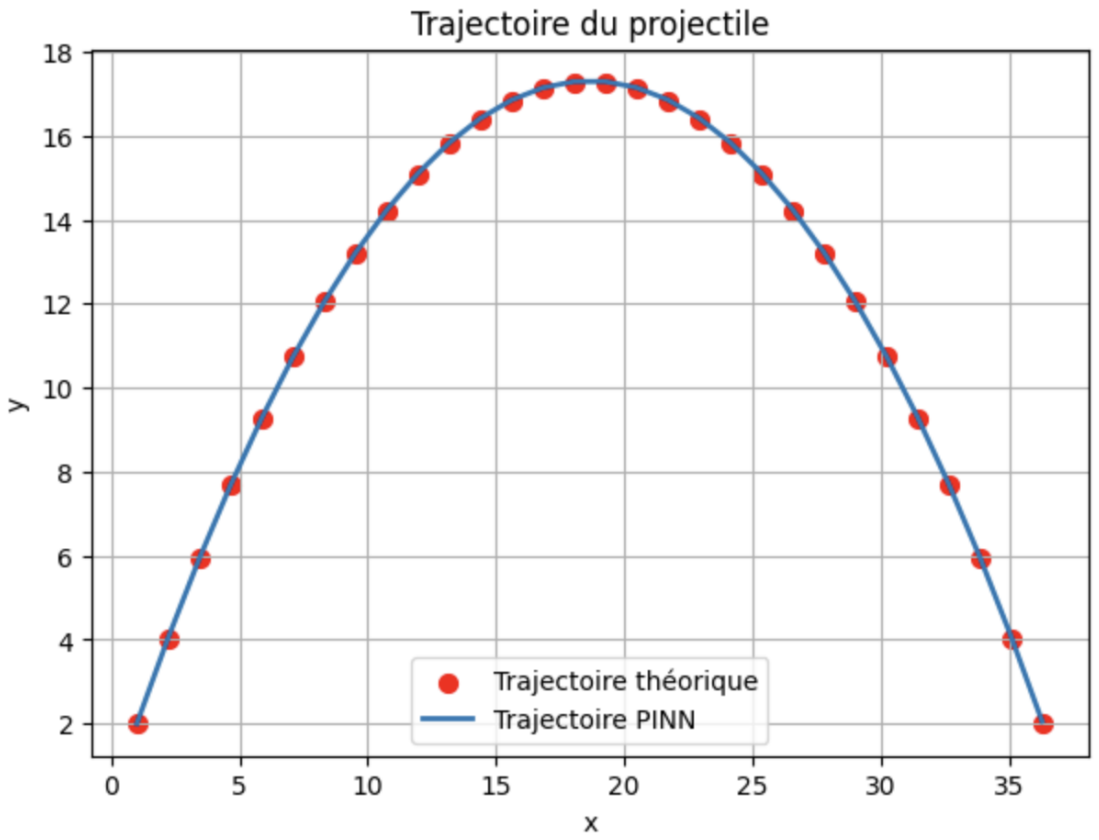
```python
# ============================
# 12. Visualisation trajectoire y(x)
# ============================

plt.figure(figsize=(7, 5))

plt.scatter(
    x_exact,
    y_exact,
    label="Trajectoire théorique",
    linewidth=2,
    color="red"
)

plt.plot(
    x_pinn,
    y_pinn,
    label="Trajectoire PINN",
    linewidth=2
)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Trajectoire du projectile")
plt.legend()
plt.grid()
plt.show()
```



Implémentation des PINNs

```
[project]
name = "pinns"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "ipykernel>=7.1.0",
    "keras>=3.13.2",
    "matplotlib>=3.10.8",
    "pandas>=3.0.0",
    "scikit-learn>=1.8.0",
    "tensorflow>=2.20.0",
    "torch>=2.10.0",
    "torchvision>=0.25.0",
    "torchviz>=0.0.3",
]
```

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from math import sin, pi
```

Implémentation des PINNs : Generate Synthetic Data

```
# Physics parameters
g = 9.8       # acceleration due to gravity
h0 = 1.0      # initial height
v0 = 13.5     # initial velocity
alpha = pi/4

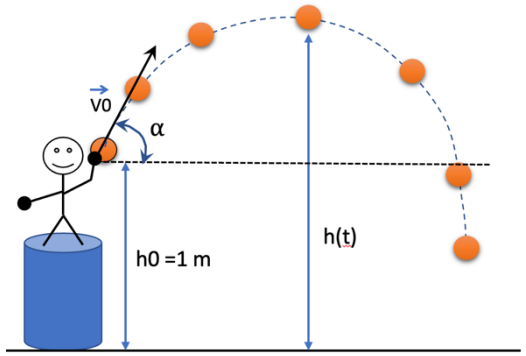
# True (analytical) solution h(t) = h0 + v0*t - 0.5*g*t^2
def true_solution(t):
    return h0 + v0*sin(alpha)*t - 0.5*g*(t**2)

# Generate some time points
t_min, t_max = 0.0, 2.0
N_data = 10
t_data = np.linspace(t_min, t_max, N_data)
```

```
# Generate synthetic "experimental" heights with noise
np.random.seed(0)
noise_level = 0.5
h_data_exact = true_solution(t_data)
h_data_noisy = h_data_exact + noise_level*np.random.randn(N_data)

# Convert to PyTorch tensors
t_data_tensor = torch.tensor(t_data, dtype=torch.float32).view(-1, 1)
h_data_tensor = torch.tensor(h_data_noisy, dtype=torch.float32).view(-1, 1)
```

```
pd.DataFrame({"t":t_data, "h_Exp":h_data_noisy, "h_Physics": h_data_exact})
```



$$\frac{dh}{dt} = V_0 \sin\alpha - g\,t$$

$$h = -\frac{1}{2} g t^2 + V_0 \sin\alpha\, t + h_0$$

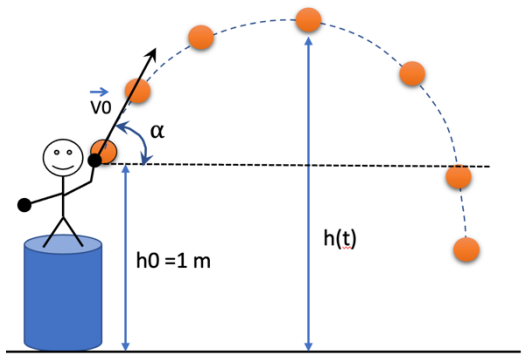
V_0 : Initial velocity at angle α

g : Acceleration due to gravity

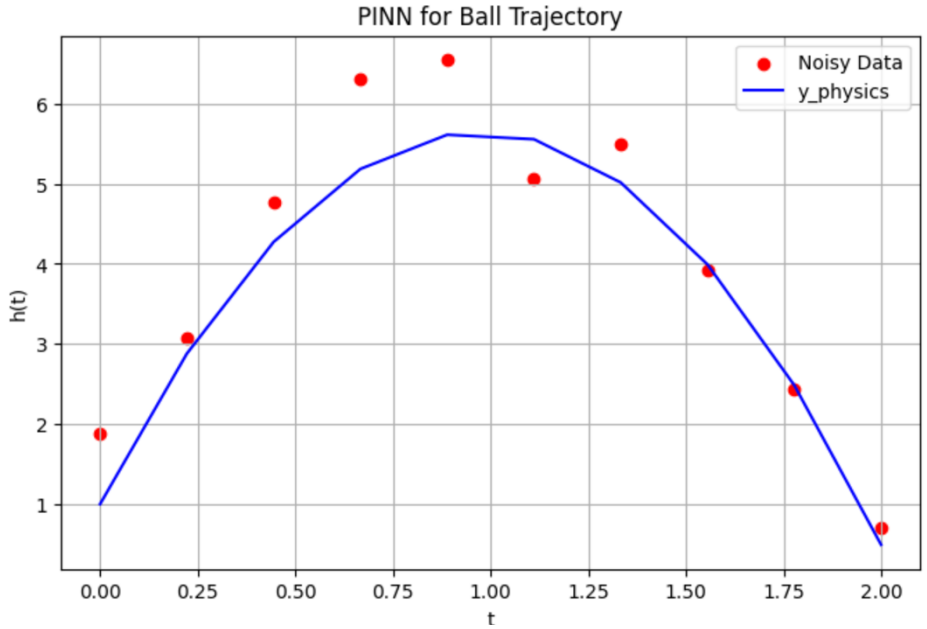
	t	h_Exp	h_Physics
0	0.000000	1.882026	1.000000
1	0.222222	3.079424	2.879345
2	0.444444	4.764108	4.274739
3	0.666667	6.306630	5.186183
4	0.888889	6.547455	5.613676
5	1.111111	5.068580	5.557219
6	1.333333	5.491855	5.016811
7	1.555556	3.916774	3.992452
8	1.777778	2.432534	2.484143
9	2.000000	0.697182	0.491883

Implémentation des PINNs : Plot Results

```
# Plot results
plt.figure(figsize=(8, 5))
plt.scatter(t_data, h_data_noisy, color='red', label='Noisy Data')
plt.plot(t_data, h_data_exact, color = 'blue', label = "y_physics")
plt.xlabel('t')
plt.ylabel('h(t)')
plt.legend()
plt.title('PINN for Ball Trajectory')
plt.grid(True)
plt.show()
```



	t	h_Exp	h_Physics
0	0.000000	1.882026	1.000000
1	0.222222	3.079424	2.879345
2	0.444444	4.764108	4.274739
3	0.666667	6.306630	5.186183
4	0.888889	6.547455	5.613676
5	1.111111	5.068580	5.557219
6	1.333333	5.491855	5.016811
7	1.555556	3.916774	3.992452
8	1.777778	2.432534	2.484143
9	2.000000	0.697182	0.491883



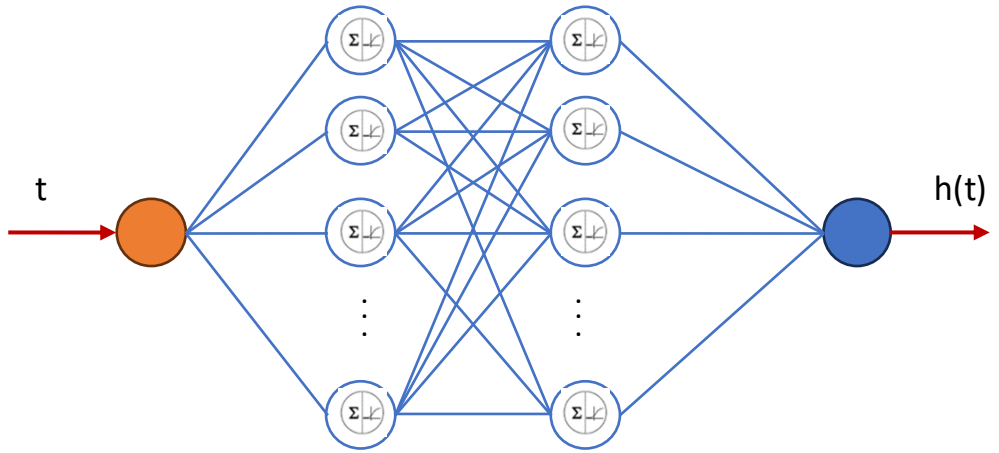
PINNs Implementation : Pytorch Neural Network Model

```
class PINN(nn.Module):
    def __init__(self, n_hidden=20):
        super(PINN, self).__init__()
        # A simple MLP with 2 hidden layers
        self.net = nn.Sequential(
            nn.Linear(1, n_hidden),
            nn.Tanh(),
            nn.Linear(n_hidden, n_hidden),
            nn.Tanh(),
            nn.Linear(n_hidden, 1)
        )

    def forward(self, t):
                return self.net(t)
```

```
model_pinns = PINN(n_hidden=30)
```

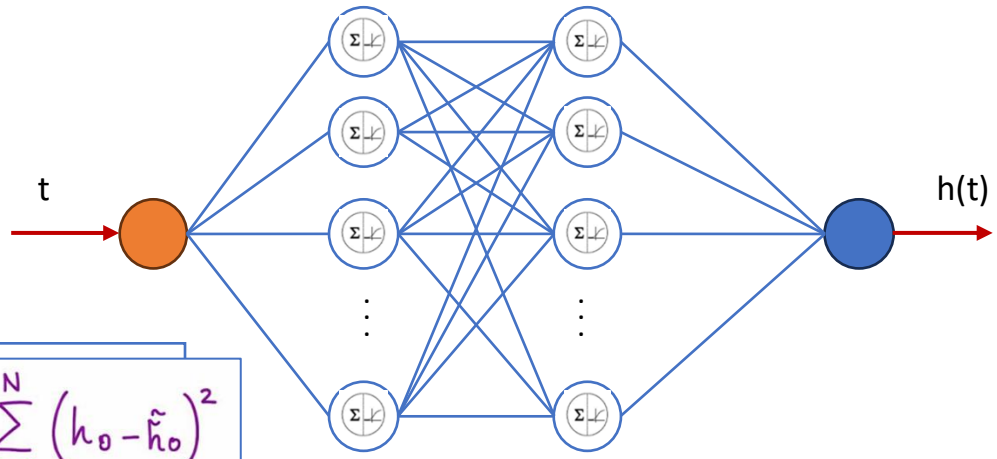
```
PINN(
  (net): Sequential(
    (0): Linear(in_features=1, out_features=30, bias=True)
    (1): Tanh()
    (2): Linear(in_features=30, out_features=30, bias=True)
    (3): Tanh()
    (4): Linear(in_features=30, out_features=1, bias=True)
  )
)
```



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh x}{\cosh x}$$

PINNs Implementation : Loss Functions

```
def derivative(y, x):
    return torch.autograd.grad(
        y, x,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]
```




```
def data_loss(model, t_data, h_data):
    h_pred = model(t_data)
    return torch.mean((h_pred - h_data)**2)
```

$$\frac{1}{N}\sum_{i=1}^{N}\left(h_0-\tilde{h}_0\right)^2$$

```
def initial_condition_loss(model):
    t0 = torch.zeros(1, 1, dtype=torch.float32, requires_grad=False)
    h0_pred = model(t0)
    return (h0_pred - h0).pow(2).mean()
```

$$\frac{1}{N}\sum_{i=1}^{N}\left(h_0-\tilde{h}_0\right)^2$$

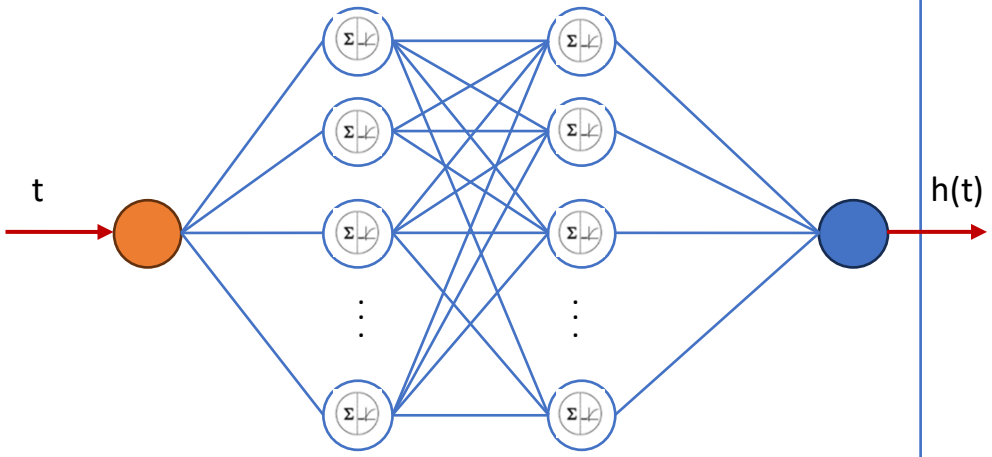
```
def physics_loss(model, t):
    t.requires_grad_(True)
    h_pred = model(t)
    dh_dt_pred = derivative(h_pred, t)
    dh_dt_true = v0 * sin(alpha) - g * t
    loss_ode = torch.mean((dh_dt_pred - dh_dt_true)**2)
    return loss_ode
```

$$\frac{1}{N}\sum_{i=1}^{N}\left(v_0\sin\alpha - g\,t_i - d\frac{\hat{h}_i}{dt}\right)^2$$

PINNs Implementation : Model Training

```
lambda_data = 2.0
lambda_ode  = 8.0
lambda_ic   = 8.0
num_epochs = 8000
print_every = 1000
```

```
def train (model, with_pinn=True):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    model.train()
    for epoch in range(num_epochs):
        optimizer.zero_grad()
                l_data = data_loss(model, t_data_tensor, h_data_tensor)
        if with_pinn == True:
            l_ode  = physics_loss(model, t_data_tensor)
            l_ic   = initial_condition_loss(model)
            loss = lambda_data * l_data + lambda_ode * l_ode + lambda_ic * l_ic
        else :
            loss =  l_data
        # Backprop
        loss.backward()
        optimizer.step()
        # Print progress
        if (epoch+1) % print_every == 0:
            print(f"Epoch {epoch+1}/{num_epochs}, "
                f"Total Loss = {loss.item():.6f}, ")
```

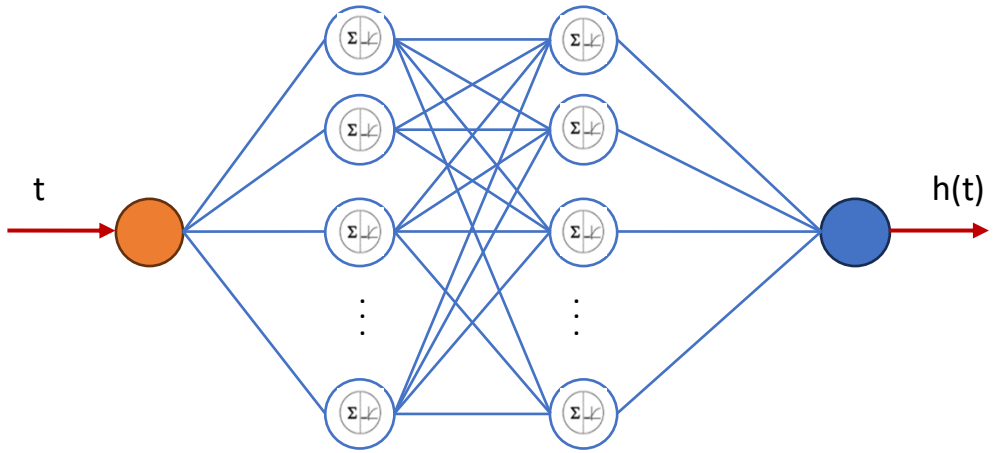


PINNs Implementation : Model Training

```
# Instantiate models
model_no_pinns = PINN(n_hidden=30)
model_pinns = PINN(n_hidden=30)
```

```
print("="*60)
print("Train Model without pinns")
print("="*60)
train(model_no_pinns, with_pinn=False)

print("="*60)
print("Train Model With Pinns")
print("="*60)
train(model_pinns, with_pinn=True)
```




```
============================================
Train Model without pinns
============================================
Epoch 1000/8000, Total Loss = 0.125508,
Epoch 2000/8000, Total Loss = 0.079400,
Epoch 3000/8000, Total Loss = 0.061639,
Epoch 4000/8000, Total Loss = 0.024623,
Epoch 5000/8000, Total Loss = 0.000163,
Epoch 6000/8000, Total Loss = 0.000000,
Epoch 7000/8000, Total Loss = 0.000000,
Epoch 8000/8000, Total Loss = 0.000000,
============================================
Train Model With Pinns
============================================
Epoch 1000/8000, Total Loss = 0.630344,
Epoch 2000/8000, Total Loss = 0.553990,
Epoch 3000/8000, Total Loss = 0.552962,
Epoch 4000/8000, Total Loss = 0.551791,
Epoch 5000/8000, Total Loss = 0.549840,
Epoch 6000/8000, Total Loss = 0.548039,
Epoch 7000/8000, Total Loss = 0.542464,
Epoch 8000/8000, Total Loss = 0.300015,
```

PINNs Implementation : Model Evaluation

model_pinns.eval()

model_no_pinns.eval()

```
t_plot = np.linspace(t_min, t_max, 100).reshape(-1, 1).astype(np.float32)
t_plot_tensor = torch.tensor(t_plot, requires_grad=True)
# Data Driven Model Prediction
h_pred_no_pinns_plot = model_no_pinns(t_plot_tensor).detach().numpy()
# Pinns Model Prediction
h_pred_pinns_plot = model_pinns(t_plot_tensor).detach().numpy()
# True solution (for comparison)
h_true_plot = true_solution(t_plot)
# Plot results
plt.figure(figsize=(8, 5))
plt.scatter(t_data, h_data_noisy, color='red', label='Noisy Data')
plt.plot(t_plot, h_true_plot, 'k--', label='Exact Solution')
plt.plot(t_plot, h_pred_pinns_plot, 'b', label='PINN Prediction')
plt.plot(t_plot, h_pred_no_pinns_plot, 'r', label='No PINNs Prediction')
plt.xlabel('t')
plt.ylabel('h(t)')
plt.legend()
plt.title('PINN for Ball Trajectory')
plt.grid(True)
plt.show()
```

